Neural Networks 206 (2027) 109802

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Beyond temporal encoding: Efficient and hierarchical model-level online adaptation for tabular distribution shift

Tianyu Wang [a,*], Mian Li [b], Xi Vincent Wang [a], Lihui Wang [a]

[a] Department of Production Engineering, KTH Royal Institute of Technology, Stockholm, Sweden
[b] Global Institute of Future Technology, Shanghai Jiao Tong University, Shanghai, China

ARTICLE INFO

Keywords:
Tabular learning
Temporal distribution shift
Online adaptation
Ensemble methods
k-nearest neighbors
Foundation models

ABSTRACT

Tabular models suffer severe performance degradation under temporal distribution shift. Existing approaches address this mainly at the feature level, for example through temporal encoding (TARS, TE) or feature modulation, but require costly end-to-end tuning and retraining for each dataset and cannot adapt to novel shift patterns at test time. A complementary *model-level* approach is proposed: a three-tier architecture inspired by prefrontal cortex plasticity and informed by a neural-manifold view of multi-timescale adaptation. The slow tier provides frozen pretrained priors (foundation models or tree ensembles), the intermediate tier performs fixed-weight ensemble blending, and the fast tier applies a novel dual-buffer *k*-nearest-neighbor correction in learned embedding space. This fast tier combines a short buffer for recent drift with a long buffer for global stability, using drift-aware blending to respond automatically to distribution change. Unlike feature-level methods, the proposed approach adapts continuously to evolving distributions without retraining. On eight TabReD benchmarks, with additional validation on thirteen temporal and domain-shift datasets, the architecture achieves the best reported RMSE on four of five TabReD regression datasets, on three of them by margins beyond run-to-run variation (reductions of up to 2.7% over the strongest baseline on each dataset) and competitive or superior AUC (0.9652, 0.8697, and 0.6373) on the three binary classification datasets. As a downstream benefit of the improved point predictions, adaptive conformal inference yields prediction intervals 3–33% narrower at equivalent 90% coverage on the regression benchmarks. All of these gains derive from the same mechanism: model-level online residual correction over frozen priors, without test-time retraining.

1. Introduction

Tabular data pervades high-stakes decision-making: from credit scoring and insurance pricing to logistics optimization and weather forecasting. Unlike images or text, tabular datasets are characterized by heterogeneous feature types, irregular inter-feature relationships, and an absence of spatial or sequential locality that could be exploited by convolutional or recurrent inductive biases (Grinsztajn et al., 2022). While gradient-boosted decision trees (GBDTs) and, more recently, deep tabular models have achieved strong performance under the standard independent and identically distributed (i.i.d.) assumption, real-world deployments face a persistent challenge: *temporal distribution shift* (Rubachev et al., 2025). As the data-generating process evolves over time (due to changing user behavior, market dynamics, seasonal effects, or policy interventions), models trained on historical data experience systematic performance degradation.

The recent emergence of tabular foundation models presents both an opportunity and a challenge for temporal adaptation. Models such as TabPFN (Hollmann et al., 2025) and LimiX (Ye et al., 2023) demonstrate strong in-context learning capabilities, achieving competitive performance across diverse datasets without task-specific training. These models produce not only predictions but also rich learned embeddings that capture semantic structure in the input space. However, their temporal robustness remains limited: like traditional models, they suffer accuracy degradation under distribution shift unless temporal context is explicitly incorporated (Helli et al., 2024). This creates an opportunity for *post-hoc* correction methods that leverage the frozen embeddings of foundation models while adapting to evolving distributions online.

Recent benchmarks such as Wild-Time (Yao et al., 2022), TableShift (Gardner et al., 2023), and TabReD (Rubachev et al., 2025) make this failure mode explicit, documenting large performance gaps between in-distribution and temporally shifted evaluation. In particular, TabReD shows that even strong pretrained tabular models can lose substantial accuracy on future time segments (Helli et al., 2024; Hollmann et al., 2025), motivating architectures that can adapt online without retraining.

* Corresponding author.
E-mail addresses: tianyuwa@kth.se (T. Wang), mianli@sjtu.edu.cn (M. Li), wangxi@kth.se (X.V. Wang), lihuiw@kth.se (L. Wang).

https://doi.org/10.1016/j.neunet.2026.109802
Received 11 March 2026; Received in revised form 28 August 2026; Accepted 24 October 2026
Available online 26 October 2026
0893-6080/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Existing approaches to temporal robustness in tabular learning mainly operate at the *feature level*, for example through temporal encodings, feature modulation, domain generalization, or test-time adaptation (Cai & Ye, 2025b; Cai et al., 2024; Jin et al., 2024; Kirichenko et al., 2023; Li et al., 2025; Pellicani & Ceci, 2025; Tancik et al., 2020; Wang et al., 2020, 2022, 2026; Xu et al., 2024; Yang et al., 2024; Zhang et al., 2023). While effective in some settings, these methods typically fix temporal structure during training, require costly retraining or gradient-based adaptation, or fail to exploit the delayed label stream available in many real-world systems. More generally, they do not provide a principled framework for coordinating adaptation across multiple timescales, from rapid correction of recent errors to stable preservation of long-term regularities.

The present work draws inspiration from a different source: the neuroscience of prefrontal cortex (PFC) plasticity. Miller and Constantinidis (2024) argue that robust adaptive behavior emerges from a hierarchy of plasticity mechanisms operating at distinct timescales, from fast neural dynamics through intermediate synaptic reconfiguration to slow structural consolidation. Crucially, these timescales are nested rather than independent: slow structural constraints shape the space within which fast adaptation operates, while persistent fast patterns gradually consolidate into slower structure. Read through a neural-manifold perspective (Perich et al., 2025), this suggests a geometric decomposition in which slow processes define a stable representational substrate, intermediate processes reconfigure task-relevant subspaces, and fast processes implement local episodic correction. This provides a natural conceptual template for temporal adaptation in tabular learning, where the central challenge is likewise to balance stability against rapid response to evolving error patterns.

Motivated by this view, a complementary *model-level* approach to temporal robustness is proposed: a three-tier architecture that performs post-hoc online correction on frozen model predictions. At a high level, the architecture combines stable pretrained priors, lightweight reconfiguration across multiple priors, and fast online correction from recent residual feedback. This decomposition is designed to capture the central multi-timescale requirement that feature-level temporal encodings and standard test-time adaptation methods leave unresolved: adaptation should be able to respond quickly to local drift without sacrificing the long-horizon structure already encoded by strong pretrained models.

In the present instantiation, this framework is realized differently for regression and classification while preserving the same high-level logic. For regression, the prior pool is centered on a tabular foundation model together with a complementary tree-based model. For classification, the prior pool is a heterogeneous mixture of tree models and pretrained neural predictors. In both cases, the architecture avoids gradient-based test-time adaptation entirely, relying instead on frozen priors, lightweight reconfiguration, and online error correction. Beyond point predictions, the regression instantiation integrates adaptive conformal inference (ACI) (Gibbs & Candès, 2021) to provide calibrated prediction intervals under temporal shift.

Evaluated on eight datasets from the TabReD benchmark (five regression, three binary classification), with further validation on thirteen additional temporal and domain-shift benchmarks, the architecture achieves RMSE reductions of 3.6–18.0% over the single-prior baseline on regression tasks. For classification, where the slow tier is instantiated as a heterogeneous ensemble of tree models (XGBoost, CatBoost, LightGBM) and pretrained neural networks (TabM, MLP) with optimized blending weights, the system achieves AUC of 0.9652, 0.8697, and 0.6373, matching the strongest end-to-end-retrained baselines on the near-stationary datasets (homesite-insurance within 0.1 pp and ecom-offers within 0.0001 AUC, i.e. statistical ties) and outperforming them where shift is pronounced (homecredit-default, +2.4 pp), at a fraction of the computational cost. The online kNN correction is the most impactful regression component (2.3–9.1% RMSE reduction), while ensemble blending and cascaded residual correction provide complementary gains. On the regression benchmarks, ACI-based prediction intervals are 3–33% narrower than those of the uncorrected prior at equivalent 90% coverage, with 1.4–2.2× more stable coverage under distribution shift. The online adaptation operates on CPU with no gradient updates to the frozen pretrained priors (its only learnable parameters are a small, optional low-rank embedding adapter, frequently disabled), achieving 23× speedup over online neural fine-tuning and 86× relative to the one-time training cost of end-to-end retraining approaches such as TARS. Extensive ablation studies validate each component's contribution.

The main contributions of this work are as follows:

- **C1.** A three-tier architecture inspired by PFC hierarchical plasticity is proposed for temporal adaptation in tabular learning, combining frozen pretrained priors, lightweight multi-prior reconfiguration, and online non-parametric correction without gradient updates to the frozen pretrained priors. This contribution is justified conceptually by the PFC/manifold mapping (Section 3) and empirically by the consistent full-system gains over single-prior and feature-level baselines (Tables 5 and 6, and the ablation in Table 7).
- **C2.** An online correction mechanism is introduced that combines dual-buffer episodic memory, drift-aware blending, and cascaded residual refinement; both theoretical and empirical analysis show that fixed ensemble weights are near-optimal for the high-correlation ($\rho > 0.95$) prior pools studied here. This is justified by the ablation, in which the fast-tier kNN correction is the most impactful regression component (Table 7), and by the fixed-weight near-optimality analysis (Section 4.2).
- **C3.** Strong empirical results are demonstrated across a full evaluation suite of twenty-one datasets, comprising eight TabReD benchmarks and thirteen additional temporal or domain-shift tasks, supporting robustness beyond a single benchmark family. The efficiency of this profile is quantified in Table 18 (CPU-only online adaptation, with 23× speedup over online fine-tuning and 86× relative to TARS's one-time training).
- **C4.** On the regression benchmarks, adaptive conformal inference yields prediction intervals 3–33% narrower at 90% coverage, with 1.4–2.2× more stable coverage under distribution shift (Section 5.4).

Here C1 and C2 are the central methodological contributions, with C3 and C4 reporting the supporting empirical evidence; the specific prior pools, embedding choices, the optional low-rank adapter, the Optuna search, and the ACI integration are interchangeable instantiation details. The constituent primitives (kNN retrieval, ensembling, conformal prediction) are established; the contribution lies in their training-free, multi-timescale composition at the model level.

The remainder of this paper is organized as follows. Section 2 surveys related work on temporal shift in tabular learning. Section 3 introduces the neuroscience foundations motivating the architecture. Section 4 details the three-tier architecture and the ACI integration. Section 5 presents experimental results, ablation studies, and uncertainty quantification analysis. Section 6 concludes with a discussion of limitations and future directions.

2. Related work

2.1. Benchmarks and empirical evidence for temporal shift

The dominant evaluation paradigm for tabular learning has long assumed independent and identically distributed (i.i.d.) data, with standard benchmarks such as UCI and OpenML driving progress on GBDT and deep learning methods alike (Grinsztajn et al., 2022; Villaizán-Vallelado et al., 2024). Recent large-scale evaluations spanning hundreds of datasets (Ye et al., 2025) further underscore the sensitivity of method rankings to benchmark design choices. However, the concept drift literature has independently developed a rich taxonomy of non-stationarity, covering shifts in input distributions, conditional label distributions, and their temporal profiles (sudden, gradual, incremental,

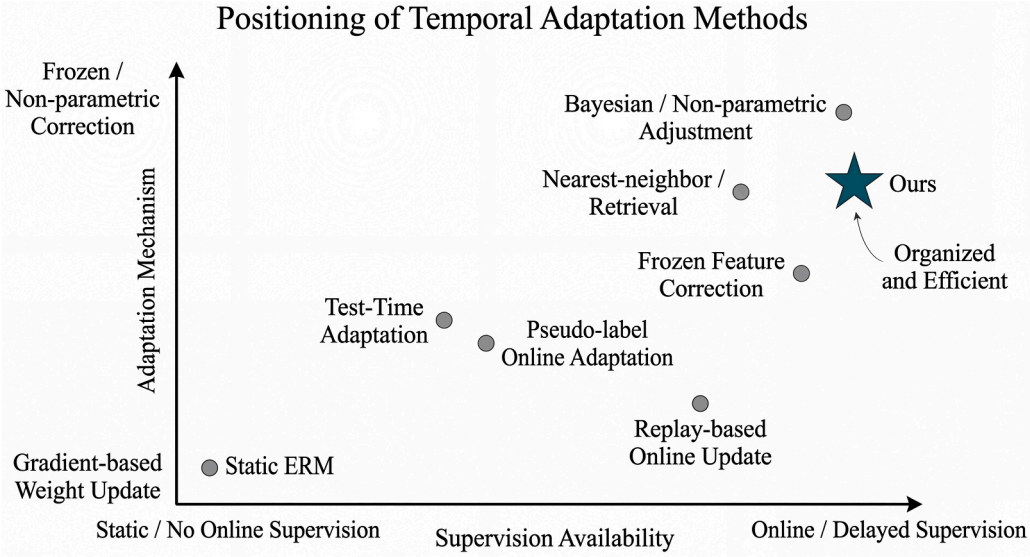

Fig. 1. Positioning of temporal adaptation methods. The proposed approach combines frozen non-parametric correction with online delayed supervision, achieving both efficiency and adaptivity.

recurring), alongside streaming evaluation protocols that impose single-pass access and delayed label constraints (Aguiar et al., 2024; Aguiar & Cano, 2024; Bayram et al., 2022).

These two communities have converged through a series of temporally-aware tabular benchmarks. Wild-Time (Yao et al., 2022) introduces discrete temporal partitions with Eval-Fix and Eval-Stream protocols, though its coarse time indices limit analysis of gradual drift. TableShift (Gardner et al., 2023) curates tasks with controlled long-horizon shifts, revealing substantial in-distribution to out-of-distribution performance gaps. Most relevant to the present work, TabReD (Rubachev et al., 2025) assembles naturally timestamped real-world datasets where temporal partitions reflect realistic deployment conditions, spanning high-frequency fluctuations to multi-year trends, and serves as the primary benchmark in the present evaluation.

Empirical studies on these benchmarks reveal a consistent finding: learned representations degrade under temporal shift in ways that static models fail to anticipate. Vela et al. (2022) document progressive declines in both accuracy and feature stability over time. Cai and Ye (2025b) show that deep tabular models tend to overfit short-term patterns while ignoring long-range temporal signals, leading to representation collapse. Pretrained tabular foundation models exhibit similar vulnerabilities unless temporal context is explicitly incorporated during inference (Hollmann et al., 2025), and multi-scale temporal representations have been advocated as a promising direction (Helli et al., 2024). Collectively, these findings motivate architectures that embed temporal adaptation directly into the prediction pipeline rather than relying on static feature representations.

2.2. Approaches to temporal robustness

Existing methods for temporal robustness in tabular learning can be organized into three broad families, each addressing a different stage of the modeling pipeline, as summarized in Fig. 1.

2.2.1. Temporal feature engineering

The most straightforward approach augments tabular inputs with explicit time-derived features. Fourier-based timestamp embeddings (Cai & Ye, 2025b; Tancik et al., 2020) project scalar time values into high-dimensional periodic representations, capturing seasonal patterns but committing to fixed frequency bases at training time. Adaptive alternatives include Faith (Li et al., 2025), which learns harmonic basis weights jointly with task objectives, and PosEncTab (Pellicani & Ceci, 2025), which imports positional encoding ideas from sequence modeling into tabular streams. DFNet (Zhang et al., 2023) applies seasonal-trend decomposition before fusing temporal and static features, while Salience Interest Option (Zhu et al., 2024) introduces salience functions for temporal abstraction across multiple timescales. More recently, SAFT (Zhang et al., 2025) integrates shift-aware anti-shift mechanisms into feature transformation, but still operates within a single-stage pipeline. TARS (Wang et al., 2026) goes further by decomposing timestamps into short-term recency, mid-term periodicity, and long-term contextual embeddings, combined with an implicit drift encoder and drift-aware routing that adaptively weights these temporal pathways. While TARS demonstrates consistent improvements across multiple backbones, it remains a plug-in temporal encoding module and does not incorporate online error correction from revealed test-time labels. Although TARS and the present work draw on the same neuroscience motivation (Miller & Constantinidis, 2024), the two approaches translate the PFC hierarchy at fundamentally different levels. TARS operates at the representation level: it decomposes temporal information into multi-scale input encodings that are fused with features before entering the backbone. The present work instead translates the hierarchy into an architectural principle, where each tier realizes a distinct mechanism of adaptation, namely structural consolidation, subspace reconfiguration, and episodic correction, rather than a distinct temporal encoding granularity. In effect, TARS modifies what goes into the model, whereas the proposed architecture modifies what comes out, enabling training-free online adaptation under the prequential protocol. A shared limitation of all these approaches is that their temporal priors are fixed after training and cannot adapt to novel shift patterns encountered at test time.

2.2.2. Domain generalization and test-time adaptation

A second family treats temporal segments as distinct domains and seeks representations that are invariant across them (Cai et al., 2024; Jin et al., 2024; Xu et al., 2024). While principled, this approach discretizes continuous temporal evolution into a finite set of environments and may miss gradual drift that does not align with predefined boundaries.

Test-time adaptation methods adjust model behavior at inference time. Unsupervised approaches optimize auxiliary objectives such as entropy minimization (Wang et al., 2020, 2022) or batch statistics alignment (Ma, 2024; Yang et al., 2024; Zhou et al., 2025) using unlabeled

test data. Last-layer retraining (LLR) (Kirichenko et al., 2023) offers a lightweight supervised alternative by updating only the output layer on recent labeled samples. Pretrained tabular foundation models (Helli et al., 2024; Hollmann et al., 2025; Ye et al., 2023) expand model capacity but remain dependent on how temporal information is integrated during in-context inference. Recent work has explored in-context learning with tabular foundation models for evolving data streams (Lourenço et al., 2025), yet these approaches still lack multi-timescale coordination.

These approaches address complementary aspects of temporal robustness but share two common gaps. First, none provides a principled framework for coordinating adaptation across multiple timescales simultaneously, from rapid correction of recent errors to stable preservation of long-term regularities. Second, most methods incur substantial computational overhead: temporal encoding methods such as TARS require full end-to-end retraining of the backbone for each dataset, test-time adaptation methods perform gradient-based optimization at inference, and domain generalization approaches multiply training cost by the number of temporal segments. In real-world settings where models must be refreshed frequently under evolving distributions, this overhead constitutes a significant practical barrier. The proposed architecture addresses both gaps through a hierarchical design that separates concerns across distinct temporal tiers; critically, requires no gradient-based retraining or fine-tuning of the pretrained models; the core test-time correction is non-parametric kNN in frozen embedding space, with an optional low-rank embedding adapter as the sole lightweight gradient-based component.

2.3. Online supervised adaptation and stream learning

A distinct line of work addresses temporal shift through online supervised methods that exploit delayed ground-truth labels arriving sequentially after prediction. This setting, formalized by the prequential (predictive sequential) evaluation protocol (Dawid, 1984), reflects the natural feedback loop in many real-world systems where outcomes are eventually observed.

Classical stream-learning algorithms were designed for this regime. Hoeffding Trees (Domingos & Hulten, 2000) perform incremental decision tree induction from streaming data, with the Hoeffding Adaptive Tree (HAT) (Bifet & Gavaldà, 2009) extending this to handle concept drift via adaptive windowing. Adaptive Random Forest (ARF) (Gomes et al., 2017) combines Hoeffding Trees with online bagging and drift detection, representing the state of the art in ensemble stream classification. However, these methods process raw input features one sample at a time and scale poorly to the high-dimensional embedding spaces used by modern deep tabular models, a limitation confirmed empirically in the experiments (Section 5.2).

Simple online correction strategies include Exponentially Weighted Averaging (EWA), which applies exponential smoothing to recent prediction errors, and Last-Layer Retraining (LLR) (Kirichenko et al., 2023), which freezes a pretrained backbone and retrains only the output layer on recent labeled data. While computationally lightweight, these methods lack the locality-aware adaptation that non-parametric approaches can provide.

Nearest-neighbor methods offer a natural mechanism for local online adaptation. Test-Time Local Training (Sun et al., 2025) retrieves training-set neighbors of each test sample and fine-tunes a local model, but requires gradient updates at inference time. Kang and Kang (2026) extend local training to tabular settings specifically, confirming the potential of instance-level adaptation while inheriting the same computational overhead. The proposed approach differs in two key respects: (i) it operates entirely in the embedding space of frozen pretrained models, avoiding any updates to the pretrained parameters; and (ii) it maintains an evolving episodic memory of *test-time* prediction errors rather than relying solely on training-set neighbors, enabling genuine adaptation to distribution shift rather than merely exploiting local training-set structure. The dual-buffer design, drift-aware blending, and cascaded residual correction further distinguish the proposed method from prior kNN-based approaches.

3. Preliminaries

The proposed architecture is grounded in the theoretical framework of Miller and Constantinidis (2024), which reconsiders the role of the prefrontal cortex (PFC) through the lens of *hierarchical timescales of learning*. This framework motivates the design by positing a unified system in which plasticity spans a continuum of timescales, ranging from milliseconds to years. To articulate the geometric constraints induced by this hierarchy, the *neural manifold* perspective is additionally adopted (Perich et al., 2025). Throughout this work, “manifold” is used in this geometric and representational sense: a structured latent space that constrains which fast adaptations are locally plausible while preserving a stable global substrate.

3.1. The continuum of plasticity

Miller and Constantinidis (2024) identify three temporal regimes of PFC plasticity, each corresponding to a distinct tier in the proposed architecture. This taxonomy is adopted to organize the three-tier architecture, and the manifold view is used to articulate the geometric implications associated with each regime. At the *microscopic timescale* (seconds), the PFC maintains information through persistent neural activity and transient population dynamics, corresponding to working memory. From a manifold perspective (Perich et al., 2025), this regime can be viewed as a trajectory evolving on a fixed, low-dimensional manifold. In the proposed model, this is instantiated by the dual-buffer episodic memory and drift-aware retrieval in the fast tier (Section 4.3). At the *mesoscopic timescale* (minutes to hours), “activity-silent” mechanisms store information via transient synaptic modifications rather than sustained spiking, supporting the formation of temporary task sets. Geometrically, this corresponds to manifold reconfiguration: rapidly reshaping the task-relevant subspace to match the current context. This is captured by ensemble blending (Section 4.2), which selectively weights different representational channels. At the *macroscopic timescale* (days to years), experience shapes the functional architecture of the PFC, namely the connectivity structure determining which representations are expressible. This corresponds to the intrinsic manifold, a stable structure constraining faster dynamics. In the proposed architecture, this structural role is implemented by the slow tier, which regularizes rapid adaptation with consolidated knowledge.

3.2. Nested interdependence

The mapping from hierarchical PFC timescales to the three-tier architecture is illustrated in Fig. 2. Following Miller and Constantinidis, the proposed model explicitly instantiates *nested interdependence* across levels. In the top-down direction, macroscopic structure in the slow tier constrains the space of admissible dynamics for the microscopic state in the fast tier. In the bottom-up direction, persistent microscopic patterns gradually induce macroscopic plasticity through systems consolidation. In the proposed architecture, this pathway is realized by an online low-rank embedding adaptation that allows persistent error patterns in the fast tier to gradually reshape the embedding manifold (Section 4.3.5). To make this mapping operational rather than analogical, Table 1 ties each PFC timescale to the corresponding manifold operation, the architectural mechanism that realizes it, and the concrete design decision it dictates. We emphasize that this mapping is an inspirational design template, not a claim of biological fidelity.

4. Methodology

The theoretical framework of Section 3 establishes that adaptive prediction under distribution shift requires the coordination of processes

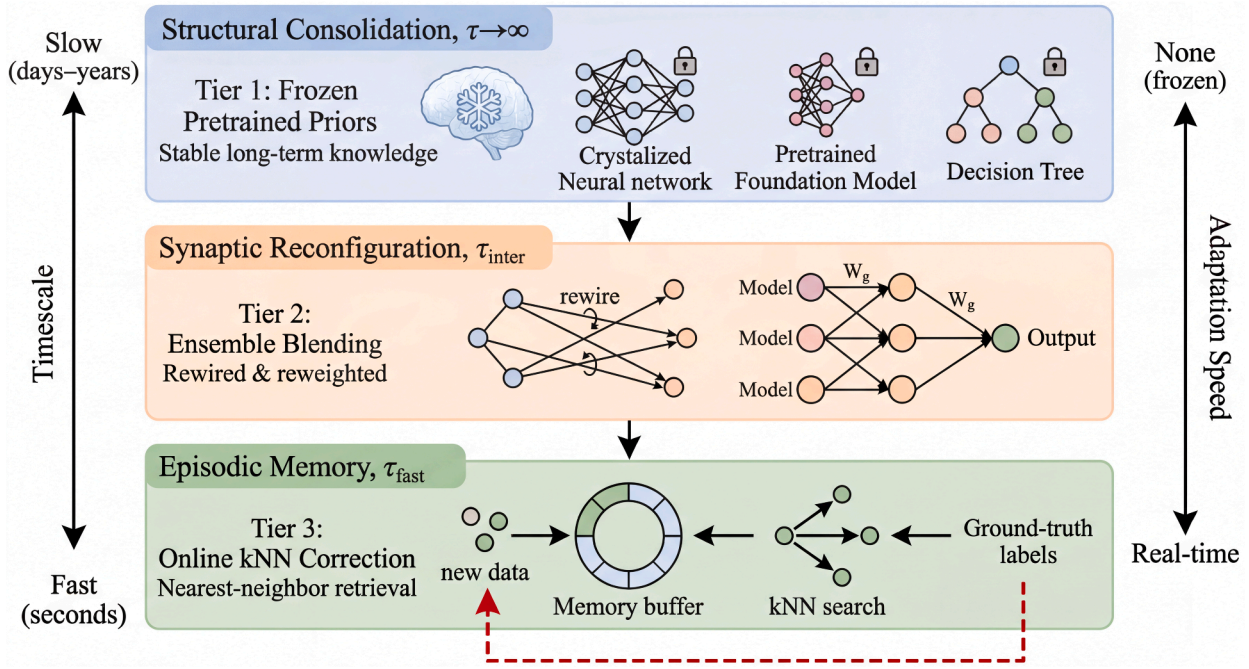

Fig. 2. Hierarchical timescales of PFC plasticity mapped to the three-tier architecture: slow tier (structural priors), intermediate tier (subspace reconfiguration), fast tier (episodic correction).

Table 1
Operational mapping from PFC plasticity timescales to architectural mechanisms and the design decisions they dictate.

PFC timescale	Manifold operation	Architectural mechanism	Design decision
Macroscopic (structural)	Stable intrinsic manifold	Slow tier: frozen priors	Keep priors frozen; training-free adaptation
Mesoscopic (task-set reconfiguration)	Subspace reconfiguration	Intermediate tier: ensemble blending	Fixed weights, near-optimal under high inter-prior correlation (Section 4.2)
Microscopic (working memory)	Trajectory on a fixed manifold	Fast tier: dual-buffer episodic memory	Short/long buffers with drift-aware blending
Nested bottom-up (consolidation)	Manifold reshaping	Online low-rank embedding adapter	Optional adapter; persistent errors reshape retrieval geometry (Section 4.3.5)

operating at distinct timescales, from slow structural consolidation to rapid episodic retrieval. This section translates that principle into a concrete computational architecture: the *Three-Tier PFC Architecture*. Each tier instantiates a specific timescale of the PFC hierarchy, and the design choices within each tier are motivated by the theoretical constraints derived above.

Formally, let $\mathcal{D} = \{(\boldsymbol{x}_t, y_t)\}_{t=1}^{T}$ denote a temporally ordered data stream whose underlying distribution $P(\boldsymbol{x}, y)$ may evolve over time. The architecture decomposes prediction into three hierarchically composed operations: (i) a frozen pretrained prior that encodes stable structural regularities, (ii) an ensemble blending layer that reconfigures representational channels, and (iii) an online episodic correction mechanism that adapts to the evolving test distribution without modifying the pretrained models' parameters. Fig. 3 provides a schematic overview. The three tiers constitute the core architecture; the specific prior pools, the embedding spaces used for retrieval, the optional low-rank adapter, the Optuna search, and the ACI integration are interchangeable instantiation choices that realize the tiers rather than core components. The invariant across instantiations is that the fast tier always retrieves in a learned embedding space selected per dataset: foundation-model embeddings for regression (tree-leaf on maps-routing, whose deployed pool is tree-plus-neural) and, for classification, tree-leaf or foundation-model embeddings chosen by the per-dataset selection of Section 5.3.3 (Section 4.3.1).

4.1. Slow tier: structural consolidation via frozen priors ($\tau \to \infty$)

The macroscopic timescale of the PFC hierarchy corresponds to structural consolidation, i.e., the formation of stable representational manifolds that encode long-timescale statistical regularities. In the proposed architecture, this role is fulfilled by a frozen pretrained model that serves simultaneously as a prediction engine and as a provider of dense learned embeddings. Crucially, the prior's parameters remain fixed throughout test-time inference: all adaptation is delegated to faster tiers. This separation ensures that the slow tier functions as a stable "intrinsic manifold" constraining the dynamics of subsequent processing stages.

The frozen prior as structural substrate. The slow prior can in principle be instantiated by any model that provides both task predictions and dense feature representations. A natural realization is a pretrained tabular foundation model, a feature-tokenizer transformer trained with a self-supervised masked reconstruction objective:

$$\mathcal{L}_{slow} = \mathcal{L}_{task} + \mathcal{L}_{recon}, \qquad \mathcal{L}_{recon} = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} (x_j - \hat{x}_j)^2, \tag{1}$$

where $\mathcal{M}$ denotes randomly masked feature positions. The reconstruction term encourages the slow tier to capture stable feature co-occurrence structure rather than task-specific shortcuts. For each sample $\boldsymbol{x}_t$, the foundation model produces a scalar prediction $\hat{y}_t$ and a dense em-

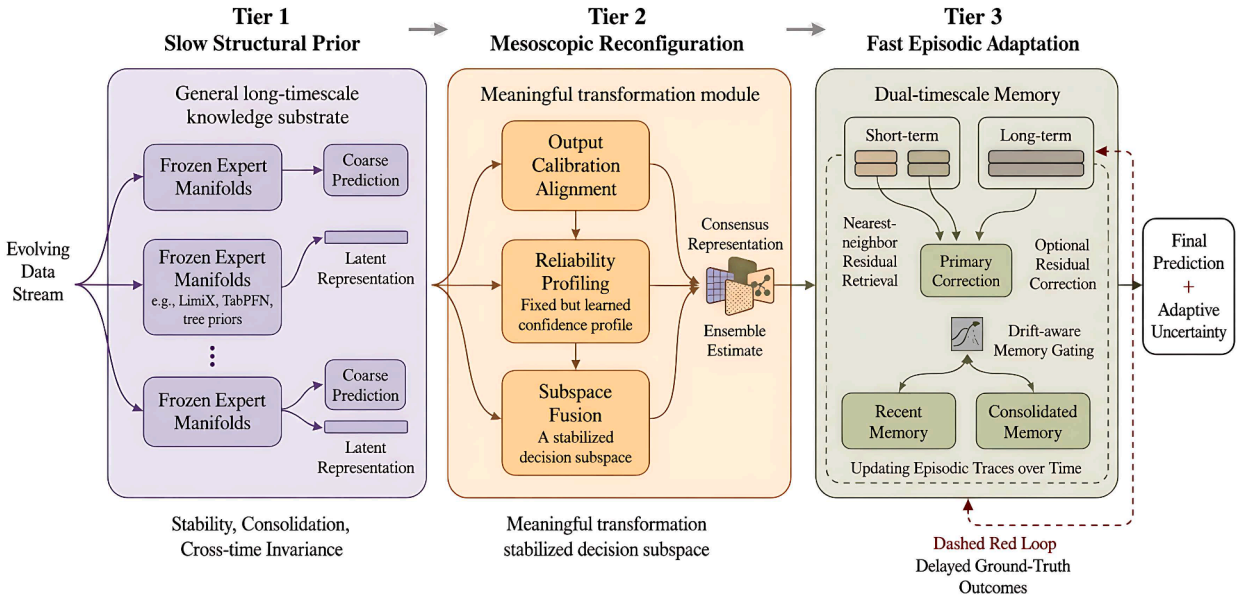

Fig. 3. Three-tier PFC architecture. **Slow tier**: frozen priors. **Intermediate tier**: ensemble blending. **Fast tier**: online kNN correction with dual-buffer episodic memory. Dashed loop: delayed label feedback.

bedding $e_i \in \mathbb{R}^d$; the latter plays a central role in the episodic correction of Level 3.

Diversity through contextual perturbation. A single frozen prior provides a point estimate, but the consolidated knowledge it encodes can be sampled from multiple perspectives. When the foundation model performs in-context learning over a window of C contextual examples, different views of the same prior arise naturally by perturbing two axes of variation: (a) temporal context policies, where varying the temporal ordering by which in-context examples are selected (e.g., sequential, time-series-aware, or temporally shuffled orderings) exposes different facets of the learned manifold; and (b) stochastic context sampling, where different random seeds yield different in-context example subsets under a fixed temporal policy, producing additional independent draws from the prior's representational capacity. Combining P temporal policies with S seed variants yields $N_{\text{neural}} = P + S$ foundation model variants, each producing a distinct prediction while sharing the same frozen parameters.

Complementary error manifolds via architectural diversity. The neural variants, though diverse in their context windows, share a common pretrained backbone and consequently exhibit high mutual correlation. To introduce a qualitatively different source of predictive information, the ensemble incorporates a structurally distinct model class, specifically a gradient-boosted decision tree (GBDT) trained on raw tabular features. The GBDT operates on a fundamentally different error manifold: its piecewise-constant decision surfaces are complementary to the smooth, attention-based representations of the foundation model. This architectural heterogeneity ensures that the ensemble spans a broader region of the hypothesis space than any single model family could achieve alone. The complete prior pool thus comprises $N = N_{\text{neural}} + 1$ variants. Specific model choices and training details are provided in Section 5.

Task-specific prior instantiation. The three-tier architecture is task-agnostic in structure but allows task-specific instantiation of each tier. For regression tasks, the foundation model (LimiX) provides both predictions and the dense embeddings that drive fast-tier correction (maps-routing excepted; Section 5). For binary classification, however, the foundation model's reconstruction-oriented embeddings do not produce discriminative neighborhoods (see ablation in Section 5.3.3). The slow tier is therefore instantiated as a heterogeneous ensemble comprising XGBoost, CatBoost, and LightGBM tree models complemented by pretrained neural networks (TabM-Modulated, MLP-Modulated, and FT-Transformer variants), with membership selected per dataset (Section 5), whose frozen predictions serve the same structural role: a stable prior that encodes consolidated knowledge of the training distribution. Crucially, the three-tier decomposition remains intact: the tree ensemble constitutes the slow tier (frozen priors), Optuna-tuned ensemble weights realize the intermediate tier (subspace reconfiguration), and kNN error correction in tree leaf embedding space provides the fast tier (episodic adaptation). This task-specific instantiation reflects a broader design principle: the PFC hierarchy prescribes the functional roles of each timescale, not the specific computational substrate that realizes them.

4.2. Intermediate tier: mesoscopic subspace reconfiguration (τ_{inter})

The mesoscopic timescale in the PFC hierarchy governs rapid subspace reconfiguration, i.e., the selective weighting of different representational channels to match the current task context. In the proposed architecture, this is realized by an ensemble blending layer that combines the N prior variants into a single estimate. Geometrically, this layer does not create a new structural manifold; instead, it reweights how the current prediction moves within the output space already spanned by the slow-tier priors.

Output manifold alignment. Because different context policies and stochastic seeds produce in-context example sets with different distributional statistics, the variant predictions may inhabit heterogeneous output manifolds. A prerequisite for meaningful combination is alignment of these manifolds. All variants are therefore rescaled to a common reference distribution via an affine transformation:

$$\hat{y}_i^{(\text{resc})} = \frac{\hat{y}_i - \mu_i}{\sigma_i + \epsilon} \cdot \sigma_{\text{base}} + \mu_{\text{base}}, \tag{2}$$

where μ_i, σ_i are the mean and standard deviation of variant i's training set predictions, and $\mu_{\text{base}}, \sigma_{\text{base}}$ are those of a designated reference variant. Rescaling is computed exclusively from training statistics, precluding information leakage.

Fixed-weight blending and the limits of dynamic gating. The blended prediction is a convex combination of the rescaled priors:

$$\hat{y}_{\text{ens}} = \sum_{i=1}^{N} w_i \, \hat{y}_i^{(\text{resc})}, \qquad \sum_{i=1}^{N} w_i = 1, \quad w_i \geq 0, \tag{3}$$

where the weights $\boldsymbol{w}$ are optimized per dataset on a held-out validation set.

A natural question is whether the weights should vary per sample, i.e., whether a dynamic gating mechanism $\boldsymbol{w}(\boldsymbol{x}_t)$ could outperform fixed weights. When the neural prior variants share a common pretrained backbone, their pairwise correlations typically exceed $\rho > 0.95$. Under such near-collinearity, a signal-to-noise argument indicates that dynamic per-sample weighting offers limited expected benefit: the signal-to-noise ratio for per-sample differentiation among highly correlated predictors scales as $\mathcal{O}(1-\rho)$, which vanishes as $\rho \to 1$. Intuitively, when two predictors agree on 95% of their variance, the residual 5% that distinguishes them is dominated by noise rather than by exploitable per-sample structure. Oracle analysis confirms this: the per-sample best-prior improvement over the ensemble mean is negligible, leaving little room for dynamic exploitation in the observed high-correlation regime. Dynamic gating could in principle still help if prior errors were conditionally structured despite high marginal correlation; the oracle analysis indicates that no such structure is exploitable for the prior pools used here. Global fixed weights are therefore near-optimal in this regime.

Ambiguity decomposition and the role of diversity. The Krogh–Vedelsby decomposition (Krogh & Vedelsby, 1994) provides an exact accounting of ensemble benefit:

$$\text{MSE}_{\text{ens}} = \sum_{i=1}^{N} w_i \, \text{MSE}_i - \underbrace{\sum_{i=1}^{N} w_i \big(\hat{y}_i - \hat{y}_{\text{ens}}\big)^2}_{\text{Ambiguity } A}, \tag{4}$$

where the ambiguity A quantifies the ensemble's diversity, i.e., the degree to which individual priors disagree with the ensemble mean. This decomposition reveals that the ensemble MSE is strictly less than the weighted average of individual MSEs by exactly the ambiguity term, establishing that diversity is not merely desirable but mathematically sufficient for ensemble improvement.

Minimum-variance optimal weights. An alternative principled weight derivation follows from portfolio theory. Given the error covariance matrix $\boldsymbol{\Sigma}$ of the N priors, the minimum-variance weights are:

$$\boldsymbol{w}^* = \frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}}, \tag{5}$$

subject to a non-negativity projection. This closed-form solution optimally exploits the correlation structure among priors: highly correlated priors receive reduced weight, while priors with unique error patterns, particularly the structurally distinct XGBoost prior, receive elevated weight. Appendix B provides empirical validation of both the ambiguity decomposition and the minimum-variance weight analysis across all datasets.

4.3. Fast tier: episodic memory via online kNN error correction (τ_{fast})

The microscopic timescale of the PFC hierarchy governs rapid, episodic adaptation, i.e., the retrieval of relevant past experiences to modulate ongoing processing. The proposed architecture instantiates this principle as an online, non-parametric correction mechanism that operates entirely in the embedding manifold induced by the frozen prior. The key property is that adaptation to distribution shift occurs without updating the frozen prior parameters; in the default setting, with the optional low-rank embedding adapter of Section 4.3.5 disabled, the correction is purely retrieval-based, drawing on an episodic memory of recently observed prediction errors while remaining constrained by the local geometry of the prior representation.

4.3.1. Embedding space normalization

Reliable nearest-neighbor retrieval requires that the embedding space be metrically well-conditioned. The foundation model's raw embeddings are whitened via PCA fitted on training data, ensuring isotropic distance computation. Let $\boldsymbol{E}_{\text{train}} \in \mathbb{R}^{N_{\text{train}} \times d}$ denote the training embeddings. The whitening transformation is:

$$\boldsymbol{W}_{\text{pca}} = \boldsymbol{V} \, \text{diag}(\lambda_1^{-1/2}, \ldots, \lambda_d^{-1/2}), \qquad \tilde{\boldsymbol{e}}_t = (\boldsymbol{e}_t - \bar{\boldsymbol{e}}_{\text{train}})\boldsymbol{W}_{\text{pca}}, \tag{6}$$

where $\boldsymbol{V}$ contains eigenvectors sorted by descending eigenvalue λ_j and $\bar{\boldsymbol{e}}_{\text{train}}$ is the training embedding mean. This normalization ensures that all embedding dimensions contribute equally to the distance metric, preventing dominant variance directions from overshadowing semantically relevant but lower-variance features.

Task-specific embedding spaces. For regression tasks, the foundation model's learned embeddings provide an effective similarity metric for kNN correction. For classification tasks, where the slow tier is instantiated as a tree ensemble (Section 4.1), the episodic memory operates in tree leaf embedding space. Given the tree ensemble's M trees, each sample $\boldsymbol{x}_t$ is mapped to its leaf index in each tree, yielding a sparse M-dimensional one-hot representation. PCA reduces this to $d_{\text{leaf}} = 64$ dimensions, producing an embedding that reflects the decision boundaries learned by the tree priors. This design preserves the architectural principle that the fast tier should operate on a manifold induced by the slow tier's learned representations; just as regression uses foundation model embeddings, classification uses the tree ensemble's structural manifold. The choice of which tree model's leaves to use (concatenated ensemble, single XGBoost, or single CatBoost) is dataset-dependent and analyzed in Section 5.3.3.

4.3.2. Dual-buffer episodic memory

A central design choice is the maintenance of two complementary episodic stores, each operating at a different temporal granularity, mirroring the distinction between stable long-term working memory and volatile recency-weighted active memory. The global episodic store $\mathcal{B}_L$ (capacity M_L) accumulates observed test samples up to a fixed capacity, providing a stable, comprehensive view of the error landscape that evolves slowly as new experiences are incorporated. The recency-weighted active memory $\mathcal{B}_S$ (capacity $M_S \ll M_L$) is a circular buffer retaining only the most recent samples, capturing local and rapidly evolving error patterns that reflect the current state of distribution shift.

Under the prequential evaluation protocol, ground-truth labels become available after each test batch, enabling buffer updates:

$$\mathcal{B}_L \leftarrow \mathcal{B}_L \cup \{(\tilde{\boldsymbol{e}}_t, r_t)\}, \qquad \mathcal{B}_S \leftarrow \text{ring_push}(\mathcal{B}_S, (\tilde{\boldsymbol{e}}_t, r_t)), \tag{7}$$

where $r_t = \hat{y}_{\text{ens},t} - y_t$ is the ensemble residual error at time t. The dual-buffer design ensures that the correction mechanism simultaneously draws on a broad error history (via $\mathcal{B}_L$) and adapts rapidly to recent distributional changes (via $\mathcal{B}_S$). When the online embedding adaptation of Section 4.3.5 is active, all buffer operations, including storage, retrieval, and distance computation, use the adapted embeddings $\tilde{\boldsymbol{e}}_t^{(\text{adapt})}$ (Eq. (16)) in place of $\tilde{\boldsymbol{e}}_t$, ensuring that the episodic memory co-evolves with the embedding geometry.

4.3.3. Retrieval via softmax-temperature kernel

For a new test sample with whitened embedding $\tilde{\boldsymbol{e}}_t$, the k nearest neighbors are retrieved independently from each buffer. Let $\mathcal{N}_k(\tilde{\boldsymbol{e}}_t; \mathcal{B})$ denote the k-nearest-neighbor set from buffer $\mathcal{B}$, with corresponding distances $\{d_j\}_{j=1}^k$ and stored errors $\{r_j\}_{j=1}^k$. Rather than uniform averaging, neighbor contributions are weighted via a softmax-temperature kernel that provides smooth control over the locality of the correction:

$$\alpha_j = \frac{\exp(-d_j/T)}{\sum_{l=1}^{k} \exp(-d_l/T)}, \tag{8}$$

where $T > 0$ is a temperature hyperparameter (with separate temperatures T_S and T_L for the short and long buffers, respectively). Low

temperatures concentrate weight on the nearest neighbors, producing highly local corrections; high temperatures yield smoother, more global estimates.

Robust aggregation via MAD clipping. Episodic memories may contain outlier errors arising from label noise or transient distributional anomalies. To ensure robustness, neighbor errors are winsorized prior to aggregation using the weighted median absolute deviation (MAD). Let $\tilde{r}$ denote the weighted median of $\{r_j\}$ and MAD = median($|r_j - \tilde{r}|$). Errors are clipped to:

$$r_j^{(\text{clip})} = \text{clip}(r_j,\ \tilde{r} - c \cdot \text{MAD},\ \tilde{r} + c \cdot \text{MAD}), \tag{9}$$

where $c > 0$ is the clipping multiplier. The buffer-specific correction is then:

$$\delta_{\mathcal{B}} = \sum_{j=1}^{k} \alpha_j\, r_j^{(\text{clip})}. \tag{10}$$

Drift-aware dual-buffer blending. A fixed blend ratio between the two buffers would impose a static trade-off between recency and stability, ignoring changes in the underlying shift dynamics. Inspired by the PFC's context-switching capability, where the prefrontal cortex dynamically gates the influence of recent versus consolidated information based on environmental volatility (Miller & Constantinidis, 2024), the blend ratio is made adaptive via an online drift signal.

Let $\{|r_t|\}$ denote the sequence of absolute prediction errors observed so far. A sliding window of length W is partitioned into two halves: recent errors $\bar{\epsilon}_{\text{recent}}$ (last $W/2$ samples) and older errors $\bar{\epsilon}_{\text{older}}$ (first $W/2$ samples). The drift signal measures the relative change in error magnitude:

$$\rho_t = \frac{\bar{\epsilon}_{\text{recent}}}{\bar{\epsilon}_{\text{older}} + \varepsilon}, \tag{11}$$

where $\varepsilon > 0$ prevents division by zero. When $\rho_t > 1$, errors are increasing, indicating active distribution shift, and the system should trust the short buffer more. The adaptive blend ratio is:

$$\omega_t = \omega_0 \cdot \sigma\big(\alpha \cdot (\rho_t - 1)\big), \tag{12}$$

where $\omega_0 \in [0, 1]$ is the base blend ratio, $\alpha > 0$ controls sensitivity to drift, and $\sigma(\cdot)$ is the sigmoid function. This formulation ensures smooth, bounded adaptation: under stable conditions ($\rho_t \approx 1$), $\omega_t \approx \omega_0/2$; under active drift ($\rho_t \gg 1$), $\omega_t \to \omega_0$, shifting weight to the short buffer. Before the window is full, ω_t defaults to $\omega_0/2$.

Proposition 1 (properties of the drift-aware blend). *The blend ratio ω_t in Eq. (12) is (i) bounded, $\omega_t \in (0, \omega_0)$; (ii) monotone increasing in the drift ratio ρ_t; and (iii) reduces exactly to the static blend $\omega_0/2$ when $\alpha = 0$ or $\rho_t = 1$, so the adaptive rule strictly generalizes a fixed-ratio baseline. Moreover, (iv) since δ_{L1} (Eq. (13)) is a convex combination of MAD-clipped buffer corrections, it is bounded by the robust per-buffer range and cannot be amplified by the blending. The proofs are immediate from the boundedness and monotonicity of the sigmoid and the properties of convex combinations.*

The window size W governs a bias–variance tradeoff in drift detection: small W yields high sensitivity to abrupt shifts but is susceptible to false positives from noise fluctuations in $\bar{\epsilon}_{\text{recent}}$; large W provides more stable estimates of ρ_t but delays response to genuine drift. Under a simple change-point model where the error distribution shifts at time t^*, the expected detection delay scales as $\mathcal{O}(W)$ while the false-positive rate under the null (no drift) decays as $\mathcal{O}(1/\sqrt{W})$, reflecting the standard tradeoff in sequential change-point detection (Tartakovsky et al., 2014). In practice, W is tuned per dataset via Optuna (Section 5), with optimized values ranging from 64 to 512 across datasets, consistent with the moderate shift dynamics in TabReD.

The corrections from the two temporal scales of episodic memory are then combined:

$$\delta_{\text{L1}} = \omega_t \cdot \delta_{\mathcal{B}_S} + (1 - \omega_t) \cdot \delta_{\mathcal{B}_L}, \tag{13}$$

The Layer 1 (L1) corrected prediction is:

$$\hat{y}_{\text{L1},t} = \hat{y}_{\text{ens},t} - s_1 \cdot \delta_{\text{L1}}, \tag{14}$$

where $s_1 \in \mathbb{R}^+$ is a scaling factor that modulates the magnitude of the online correction. The correction is activated only after a sufficient number of samples have accumulated in the episodic store, ensuring that the neighbor estimates are statistically reliable; the specific warmup threshold is reported in Section 5.

4.3.4. Hierarchical error decomposition via cascaded L2 correction

The L1 correction reduces the ensemble's residual errors but may itself introduce systematic biases, for instance, consistent over- or under-correction in particular regions of the embedding space. This motivates a hierarchical error decomposition: a second kNN pass (Layer 2) is applied to correct the residuals of the first correction. This cascaded structure is theoretically grounded in the framework of stage-wise additive modeling (Friedman, 2001; Hastie et al., 2009): the full prediction $\hat{y}_t = \hat{y}_{\text{ens}} - s_1\delta_{\text{L1}} - s_2\delta_{\text{L2}}$ is an additive expansion where each stage greedily minimizes the residual of the previous stage. Just as gradient boosting iteratively fits weak learners to pseudo-residuals, the cascaded kNN fits non-parametric corrections to the residual error landscape. The key difference is that the "weak learners" are online kNN estimators operating in embedding space rather than decision trees fitted to input features, enabling adaptation to distribution shift without retraining.

Let $r_t^{(1)} = \hat{y}_{\text{L1},t} - y_t$ denote the L1 residual. After ground truth is observed, L2 maintains its own dual-buffer system $\mathcal{B}_L^{(2)}, \mathcal{B}_S^{(2)}$ storing $(\tilde{e}_t, r_t^{(1)})$ pairs. At inference, L2 queries its buffers and computes:

$$\hat{y}_{\text{L2},t} = \hat{y}_{\text{L1},t} - s_2 \cdot \delta_{\text{L2}}, \tag{15}$$

where δ_{L2} is computed analogously to Eqs. (8)–(13) but with independently tuned hyperparameters. The cascaded structure permits each layer to specialize: L1 captures the dominant error signal, while L2 addresses the residual structure that L1 leaves behind. All hyperparameter details, including the two-stage optimization procedure, are provided in Section 5. The cascade is stable by construction: s_2 acts as a boosting-style shrinkage, and the L2 corrections are MAD-clipped and computed by the same bounded convex-blend kernel as L1, so the second stage only refines the residual of L1; Proposition 2 below makes this precise.

A second non-parametric pass is not universally beneficial: when the L1 residuals are already close to white noise, fitting them with a further kNN estimator mostly captures sampling noise and can increase error. L2 is therefore wrapped in an *adaptive gate*. During the prequential stream the gate maintains an exponential moving average of L2's contribution to squared error, $\Delta_t = (r_t^{(1)})^2 - (r_t^{(2)})^2$ with $r_t^{(2)} = \hat{y}_{\text{L2},t} - y_t$, and permanently disables the layer once this average is consistently negative (i.e., L2 increases error on a sustained majority of recent batches). This renders L2 a safe, opt-out refinement, retained only on datasets where it demonstrably helps. The gate has been part of the system since before the reported results and is enabled by default; we make its description explicit here.

Proposition 2 (stability of the cascaded L2 correction). *Let $\tilde{r}_{\mathcal{B},t}^{(2)}$ and $\text{MAD}_{\mathcal{B},t}^{(2)}$ denote the weighted median and the median absolute deviation of the L1 residuals retrieved from buffer $\mathcal{B} \in \{\mathcal{B}_S^{(2)}, \mathcal{B}_L^{(2)}\}$ at query t, and let c_2 be the L2 clipping multiplier (Eq. (9)). Then: (i) bounded perturbation: $|\hat{y}_{L2,t} - \hat{y}_{L1,t}| = s_2\,|\delta_{L2}| \le s_2 \max_{\mathcal{B}}\big(|\tilde{r}_{\mathcal{B},t}^{(2)}| + c_2\,\text{MAD}_{\mathcal{B},t}^{(2)}\big)$; the second stage can never move a prediction by more than s_2 times a robust estimate of the L1 error scale observed in the retrieved neighborhood. (ii) second-order worst case: since $r_t^{(2)} = r_t^{(1)} - s_2\,\delta_{L2}$, the change in expected squared error is $\mathbb{E}[(r_t^{(2)})^2] - \mathbb{E}[(r_t^{(1)})^2] = -2s_2\,\mathbb{E}[\delta_{L2}\,r_t^{(1)}] + s_2^2\,\mathbb{E}[\delta_{L2}^2]$. In the white-noise regime motivating the gate ($\mathbb{E}[\delta_{L2}\,r_t^{(1)}] = 0$), the worst-case inflation is therefore $s_2^2\,\mathbb{E}[\delta_{L2}^2]$, second order in the shrinkage, whereas exploitable residual structure ($\mathbb{E}[\delta_{L2}\,r_t^{(1)}] > 0$) reduces the error at first order in s_2. (iii) graceful opt-out: as $s_2 \to 0$ the cascade reduces continuously to the L1 predictor, and under the adaptive gate a sustained realization of the harmful regime permanently disables the layer, so the cascade cannot accumulate unbounded*

degradation along the stream. The proofs are one line each: (i) δ_{L2} is a convex combination, over retrieved neighbors and the two buffers, of residuals clipped to $[\tilde{r}^{(2)} - c_2\,\mathrm{MAD}^{(2)}, \tilde{r}^{(2)} + c_2\,\mathrm{MAD}^{(2)}]$; (ii) expand the square; (iii) follows from Eq. (15) and the gate rule.

Scope of the theoretical guarantees. The formal results in this paper are deliberately component-level. Propositions 1 and 2 establish safety properties of the two adaptive mechanisms of the fast tier (boundedness, monotonicity, graceful static fallback, and a second-order worst case for the cascade), the change-point analysis above characterizes the detection-delay/false-positive tradeoff of the drift statistic, and the fixed-weight analysis of Section 4.2 bounds the expected benefit of dynamic gating in the observed high-correlation regime. Together these results guarantee that the online correction cannot destabilize the frozen prior: every corrective term is a bounded, robustly clipped perturbation with a continuous opt-out. They do not, however, constitute an end-to-end analysis of the composed system; no regret or convergence guarantee is claimed for the full pipeline, whose component interactions are validated empirically through the ablations of Section 5.3 rather than analytically. Deriving stream-level guarantees, such as regret bounds for dual-buffer kNN residual correction under explicit drift assumptions, remains an open problem.

4.3.5. Bottom-up consolidation via online embedding adaptation

The PFC hierarchy is not strictly top-down: Miller and Constantinidis (2024) emphasize that persistent fast-timescale patterns gradually reshape slow structural representations through systems consolidation. In the proposed architecture, this principle is realized by allowing the episodic correction signal to gradually adapt the embedding space in which neighbors are discovered. Although this mechanism resides in the fast tier, its effective timescale is intermediate, governed by a small learning rate η and strong regularization λ, placing it between rapid episodic retrieval and the static structural prior of the slow tier. This cross-scale coupling is precisely the nested interdependence identified in Section 3: persistent error patterns in the fast tier serve as a gradient signal that slowly reshapes the structural substrate, closing the loop between bottom-up consolidation and top-down constraint.

Specifically, the whitening transformation of Eq. (6) is augmented with a low-rank online adaptation. Let $\boldsymbol{U} \in \mathbb{R}^{d\times r}$ and $\boldsymbol{V} \in \mathbb{R}^{d\times r}$ (with $r \ll d$) be adaptation matrices, initialized in the standard low-rank-adapter fashion (Hu et al., 2022): $\boldsymbol{U}$ is drawn from a small zero-mean Gaussian with $\sigma_0 = 1/\sqrt{3d}$, the standard-deviation equivalent of the Kaiming-uniform down-projection initialization of the original LoRA implementation ($d = 192$ gives $\sigma_0 \approx 0.042$), and $\boldsymbol{V} = \boldsymbol{0}$. The product $\boldsymbol{U}\boldsymbol{V}^\top$ is therefore zero at initialization, so the adapter starts as an exact identity, yet it does not remain one: since $\nabla_{\boldsymbol{V}}\mathcal{L} \propto \boldsymbol{U} \neq \boldsymbol{0}$, the first revealed labels move $\boldsymbol{V}$ away from zero and activate the adaptation. (Initializing *both* factors to zero would instead make the origin a stationary point of Eq. (17), permanently disabling the adapter.) The adapted embedding is:

$$\tilde{\boldsymbol{e}}_t^{(\mathrm{adapt})} = \tilde{\boldsymbol{e}}_t \cdot (\boldsymbol{I} + \boldsymbol{U}\boldsymbol{V}^\top), \tag{16}$$

where $\tilde{\boldsymbol{e}}_t$ is the PCA-whitened embedding from Eq. (6). The low-rank structure constrains the adaptation to a r-dimensional subspace, preventing overfitting to noise while allowing the embedding geometry to evolve.

After each batch of ground-truth labels is revealed, $\boldsymbol{U}$ and $\boldsymbol{V}$ are updated via stochastic gradient descent on the prediction error:

$$\boldsymbol{U} \leftarrow \boldsymbol{U} - \eta\Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{U}} + \lambda \boldsymbol{U}\Big), \qquad \boldsymbol{V} \leftarrow \boldsymbol{V} - \eta\Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{V}} + \lambda \boldsymbol{V}\Big), \tag{17}$$

where η is the learning rate, λ is a weight decay coefficient, and $\mathcal{L}$ is the batch prediction loss. Since the kNN retrieval operation is non-differentiable (the neighbor set $\mathcal{N}_k$ is a discrete function of the embedding), the gradient $\partial\mathcal{L}/\partial\boldsymbol{U}$ is computed by treating the neighbor set as fixed for the current batch and differentiating only through the distance-weighted aggregation (Eqs. (8)–(10)) and the embedding transformation (Eq. (16)). Concretely, the loss $\mathcal{L} = \frac{1}{B}\sum_t(\hat{y}_t - y_t)^2$ depends on $\boldsymbol{U}, \boldsymbol{V}$ through the adapted query embedding $\tilde{\boldsymbol{e}}_t^{(\mathrm{adapt})}$, which affects the softmax distances $d_j = \|\tilde{\boldsymbol{e}}_t^{(\mathrm{adapt})} - \tilde{\boldsymbol{e}}_j^{(\mathrm{adapt})}\|^2$ and hence the neighbor weights α_j. This "frozen-neighbor" approximation is analogous to the straight-through estimator used in discrete optimization (Bengio et al., 2013) and is justified by the observation that small perturbations to the embedding rarely change the neighbor set when k is moderately large. In practice, this yields stable gradients that reshape the embedding geometry so that more informative neighbors are retrieved in subsequent queries.

This mechanism implements bottom-up consolidation: persistent error patterns in the fast tier's episodic corrections, indicating that the current embedding manifold is misaligned with the evolving data distribution, gradually reshape the structural substrate. The adaptation is intentionally slow (small η, strong regularization) to preserve the stability of the embedding space while incorporating the informational content of recent experience. Consistent with this stability-first design, the deployed search range for η (10^{-5}–10^{-2}, Table 4) is deliberately conservative: larger step sizes produce proportionally larger adaptations of the retrieval geometry, but move the update toward its stability boundary on the stream, and all results reported in this paper use the conservative range. Appendix E verifies the adapter's activation, quantifies its paired effect, and indicates additional headroom at larger, still-stable step sizes. The adapted embeddings $\tilde{\boldsymbol{e}}_t^{(\mathrm{adapt})}$ are used both for kNN queries and for storage in the episodic buffers, ensuring that the consolidation effect propagates through the entire correction pipeline.

Complete system equation. The overall architecture composes all three tiers into a single prediction:

$$\hat{y}_t = \underbrace{\sum_{i=1}^{N} w_i \hat{y}_i^{(\mathrm{resc})}}_{\text{Tier 2: Ensemble}} - \underbrace{s_1 \cdot \delta_{\mathrm{L1}}}_{\text{Tier 3, L1}} - \underbrace{s_2 \cdot \delta_{\mathrm{L2}}}_{\text{Tier 3, L2}}, \tag{18}$$

where δ_{L1} and δ_{L2} operate in the adapted embedding manifold of Eq. (16) with drift-aware buffer blending (Eq. (12)). In this work, the framework is instantiated with two task-specific configurations: for regression, a tabular foundation model (LimiX) across six contextual variants plus XGBoost provides the prior pool, with fast-tier correction operating in the foundation model's embedding manifold (maps-routing, whose deployed pool is tree-plus-neural, instead uses tree-leaf embeddings; Section 5); for classification, a heterogeneous ensemble of tree models (XGBoost, CatBoost, LightGBM) and pretrained neural networks (TabM, MLP, FT-Transformer; dataset-specific membership, Section 5) serves as the prior, with fast-tier correction operating in tree-leaf embedding space (or, where the per-dataset selection of Section 5.3.3 favors it, a foundation-model embedding).

This decomposition cleanly separates the roles of each timescale: the slow prior provides a stable predictive substrate, ensemble blending reconfigures representational channels, the episodic correction adapts to the evolving test distribution, and bottom-up consolidation gradually reshapes the embedding manifold, all without modifying any pretrained parameters at test time.

4.4. Uncertainty quantification via adaptive conformal inference

Beyond point predictions, reliable uncertainty estimates are essential for deployment under non-stationary data distributions. The three-tier prediction system is augmented with adaptive conformal inference (ACI) (Gibbs & Candès, 2021), an online method for constructing prediction intervals that maintains valid coverage under distribution shift. ACI is included to assess whether improved online point prediction translates into narrower calibrated intervals; it is not required by the three-tier architecture.

4.4.1. Global ACI: online residual calibration

The online kNN correction naturally produces a stream of absolute residuals $|r_t| = |\hat{y}_t - y_t|$ that serve as nonconformity scores. A sliding window $\mathcal{W}_{\text{conf}}$ of the most recent $W_{\text{conf}} = 1000$ scores (we write W_{conf} for the conformal calibration window to distinguish it from the drift-detection window W of Eq. (11)) provides a local estimate of the prediction error distribution that adapts as the data distribution evolves. Standard split conformal prediction fixes the miscoverage rate α and computes the $(1-\alpha)$-quantile of calibration scores to form prediction intervals. Under distribution shift, this fixed quantile becomes miscalibrated, leading to coverage degradation. ACI addresses this by maintaining a time-varying miscoverage rate α_t updated after each prediction:

$$\alpha_{t+1} = \text{clip}\big[\alpha_t + \gamma\big(\alpha_{\text{target}} - \mathbf{1}\{Y_t \notin \hat{C}_t\}\big),\ 0.001,\ 0.5\big], \tag{19}$$

where $\gamma > 0$ is a learning rate and $\hat{C}_t = [\hat{y}_t - q_t,\ \hat{y}_t + q_t]$ is the prediction interval, with q_t being the $(1-\alpha_t)$-quantile of scores in $\mathcal{W}_{\text{conf}}$. When the model fails to cover the true value ($\mathbf{1}\{\cdot\} = 1$), α_t decreases, causing future intervals to widen; when coverage succeeds, α_t increases, tightening intervals. This feedback loop tracks the evolving difficulty of prediction without requiring explicit shift detection.

Within the proposed architecture, ACI operates as a lightweight post-processing layer atop the full system prediction $\hat{y}_t$ from Eq. (18). The only additional computation is maintaining the sliding window of residuals and updating a single scalar α_t, which is negligible relative to the kNN queries. Crucially, the quality of the underlying point predictions directly determines the tightness of the resulting intervals: more accurate predictions produce smaller nonconformity scores, yielding narrower intervals at equivalent coverage. This provides a principled way to quantify the uncertainty reduction achieved by each tier of the architecture.

4.4.2. Local kNN conformal extension for heteroscedastic uncertainty

While global ACI improves marginal coverage control under shift, it still produces largely homoscedastic intervals, i.e., intervals whose width is driven by the global residual distribution rather than the local difficulty of a specific test point. However, prediction difficulty varies across the input space: some regions may remain well covered by the training distribution while others experience pronounced shift. To capture this heterogeneity, ACI is extended with local nonconformity scores derived from the same kNN infrastructure used for point prediction correction.

For each test point $\boldsymbol{x}_t$, a conformal buffer $\mathcal{B}_{\text{conf}}$ is maintained containing recent embedding–residual pairs $\{(\tilde{\boldsymbol{e}}_j, |r_j|)\}$, capped at $B_{\text{conf}} = 5000$ entries. At prediction time, the $k_{\text{conf}} = 50$ nearest neighbors of $\tilde{\boldsymbol{e}}_t$ are retrieved in the whitened embedding space, and the local quantile is computed as

$$q_t^{\text{local}} = \text{Quantile}_{1-\alpha_t}\Big(\{|r_j| : j \in \mathcal{N}_{k_{\text{conf}}}(\tilde{\boldsymbol{e}}_t)\}\Big), \tag{20}$$

where $\mathcal{N}_{k_{\text{conf}}}(\tilde{\boldsymbol{e}}_t)$ denotes the k_{conf} nearest neighbors in $\mathcal{B}_{\text{conf}}$. The prediction interval becomes $\hat{C}_t = [\hat{y}_t - q_t^{\text{local}},\ \hat{y}_t + q_t^{\text{local}}]$, with α_t still updated via Eq. (19). This extension yields heteroscedastic intervals: regions where neighbors exhibit large residuals receive wider intervals, while well-predicted regions receive tighter ones. The main benefit is improved conditional coverage, i.e., coverage that is approximately valid not only marginally across all test points but also within subsets defined by the input features.

5. Experiments

5.1. Datasets and implementation details

The proposed architecture is evaluated on a total of 21 datasets spanning temporal, domain, and demographic distribution shifts. The primary evaluation uses eight tabular prediction datasets with temporal distribution shift from the TabReD benchmark (Rubachev et al., 2025), among them maps-routing (a large-scale routing regression task with 984 raw features) and homecredit-default (a large-scale credit default prediction task). These datasets span both regression and binary classification (Table 2). To assess generalization, the architecture is further evaluated on 13 additional benchmarks covering temporal shifts (ACS surveys, credit card fraud, CIC-IDS-2017, College Scorecard, IEEE-CIS Fraud, Bike Sharing), domain shifts (Sepsis, Diabetes Readmission, BRFSS Diabetes), and a clinical case study (MIMIC-IV ICU mortality prediction with 235 features Johnson et al., 2023); these datasets are summarized in Section 5.5.

Table 2
Dataset summary. All datasets use temporal train/test splits from TabReD.

Dataset	Abbr.	Task	Test Size
Cooking-time	CT	Regression (RMSE↓)	41,648
Sberbank-housing	SH	Regression (RMSE↓)	4,647
Delivery-eta	DE	Regression (RMSE↓)	36,927
Weather	WE	Regression (RMSE↓)	40,840
Homesite-insurance	HI	Classification (AUC↑)	16,295
Ecom-offers	EO	Classification (AUC↑)	26,455
Maps-routing	MR	Regression (RMSE↓)	59,951
Homecredit-default	HD	Classification (AUC↑)	56,001

All datasets are processed under the prequential (test-then-train) protocol shown in Fig. 4: each test sample is predicted before its label is revealed, and the label is subsequently used to update the kNN buffers. This simulates realistic deployment conditions under temporal shift.

Fig. 5 provides a representative view of the distribution-shift regimes encountered in TabReD, contrasting a strongly shifted dataset (sberbank-housing) with a milder case (cooking-time).

Prior Instantiation. The foundation model is instantiated as LIMIX in frozen inference mode with context window $C = 3{,}000$ ($C = 10{,}000$ for weather). Six LIMIX variants span three temporal policies (`idx`, `ts`, `tn`) and three seed variants, plus one XGBoost model trained on raw features with 5-fold cross-validated predictions. For maps-routing (984 features), inputs are PCA-reduced to 128 dimensions prior to cache generation. This reduction degrades the LIMIX priors on that dataset, so the deployed maps-routing configuration replaces the LIMIX pool with a tree-plus-neural prior pool (XGBoost, LightGBM, and modulated MLP and TabM variants). Among the regression datasets, maps-routing is the only one whose slow tier departs from the LIMIX-plus-XGBoost family; the classification instantiation uses a tree-plus-neural pool of its own by design (Section 4.1). This is also why the first three columns of Table 11 are not applicable to maps-routing.

Ensemble Configuration. The pool composition is dataset-specific: seven priors (six LIMIX variants plus XGBoost) on weather, delivery-eta, and sberbank-housing, eight on cooking-time (an additional `ts`-seed variant), four tree/neural priors on maps-routing and ecom-offers, eight on homesite-insurance, and two on homecredit-default. The exact memberships and tuned weights of every deployed pool are recorded in the released per-dataset configuration files (see the Data and Code Availability statement). Inter-correlation among LIMIX variants exceeds 0.95, while the XGBoost–LIMIX correlation ranges from 0.658 to 0.981, confirming the value of architectural heterogeneity.

kNN Configuration. The long episodic buffer has capacity $M_L = 40{,}960$ and the short buffer has capacity $M_S = 256$ by default. The kNN correction (Eq. (13)) activates only after the long buffer accumulates at least 100 labeled samples, ensuring statistically reliable neighbor estimates. PCA whitening is fitted on training-set embeddings and applied to all subsequent embeddings before distance computation.

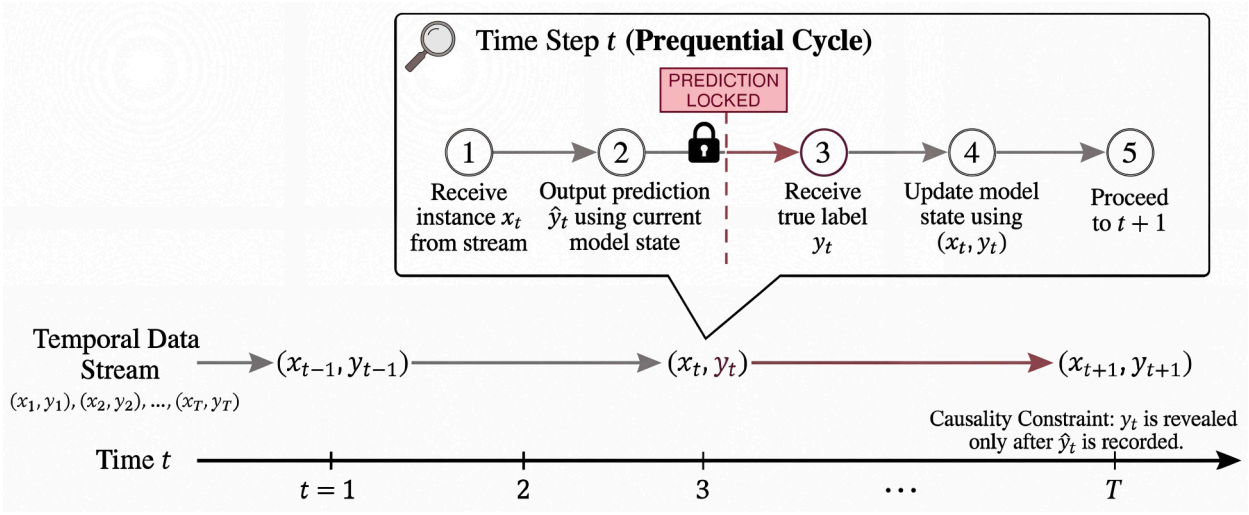

Fig. 4. Prequential evaluation protocol. At each step t: (1) receive instance x_t, (2) output prediction $\hat{y}_t$, (3) receive true label y_t, (4) update model state.

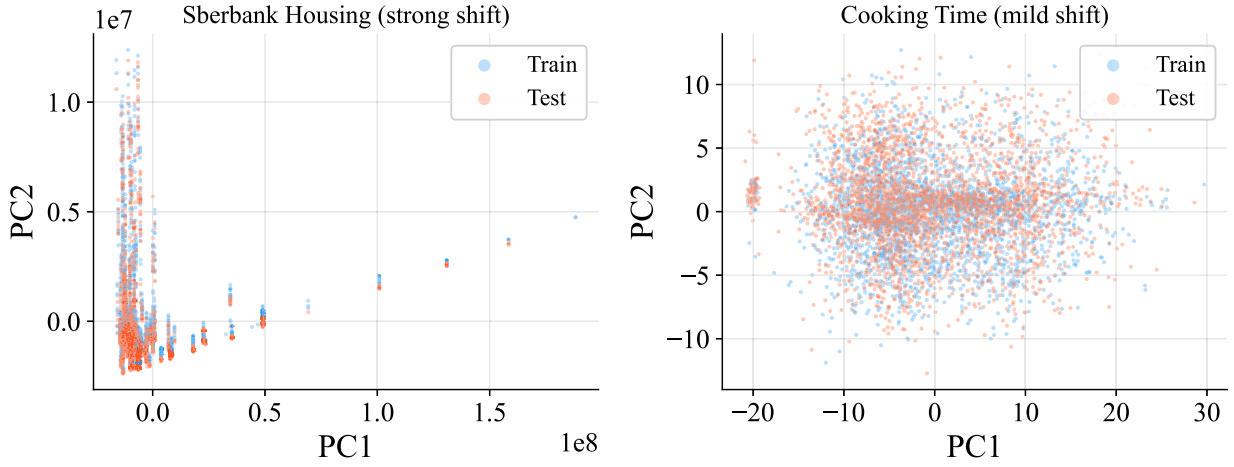

Fig. 5. PCA projection of train vs. test features. **Left:** Sberbank-housing shows pronounced distribution shift. **Right:** Cooking-time shows milder shift with substantial overlap.

Table 3
Fixed (non-tuned) hyperparameters held constant across all datasets. The per-dataset tuned hyperparameters and their search ranges are listed separately in Table 4.

Hyperparameter	Default	Role
Long buffer capacity M_L	40,960	Episodic memory size (global stability)
L1 activation threshold	100 samples	Activate correction once enough labels accrue
Ensemble size N	2–8 (dataset-specific)	Number of frozen priors; see Section 5
LimiX context window C	3,000 (10,000 weather)	Foundation-model in-context size
Embedding-adapter init	$\boldsymbol{U} \sim \mathcal{N}(0, \sigma_0^2)$, $\boldsymbol{V} = \boldsymbol{0}$; $\sigma_0 = 1/\sqrt{3d} \approx 0.0417$	Adapter starts as identity ($\boldsymbol{U}\boldsymbol{V}^\top = \boldsymbol{0}$) but trainable
PCA whitening	on (train-fit)	Embedding normalization

Hyperparameter Optimization. All per-dataset hyperparameters are optimized via Optuna Bayesian search. A fast NumPy-based simulation of the complete kNN pipeline enables approximately 10 seconds per trial, making large-scale search feasible. This search is performed on a held-out *temporal validation stream* (the chronologically last block of the training portion of each dataset) and never on the reported test stream. The NumPy prequential simulation and the optimization objective below are evaluated on this validation stream only; the selected hyperparameters are then frozen, and the reported numbers are produced by a single prequential pass over the test stream, whose labels are revealed only after each prediction (test-then-train) and are never used for hyperparameter selection. Optimization is carried out in two stages. In the first stage (L1), the ensemble weights $\mathbf{w}$, L1 kNN parameters (k, T_S, T_L, s_1, c, M_S), drift-aware blending parameters (ω_0, α, W), and embedding adaptation parameters (r, η, λ) are jointly optimized over 300 Optuna trials. In the second stage (L2), with the L1 parameters frozen, the L2-specific parameters (k_2, T_2, s_2, c_2, ω_2, $M_{S,2}$, warmup threshold) are optimized over an additional 200 trials. The optimization objective is RMSE for regression tasks and $1 - \text{AUC}$ for classification tasks. Classification datasets use wider parameter ranges (e.g., $s_1 \in [0.0, 2.0]$ and log-scale temperatures reaching $T = 50$; Table 4) to accommodate the different loss geometry.

Table 4 summarizes the key hyperparameters and their typical search ranges.

5.2. Main results and comparison

Tables 5 and 6 compare the best results of the proposed method with published baselines on TabReD, including: (i) standard tabular models (Rubachev et al., 2025); (ii) temporal adaptation methods (TARS (Wang et al., 2026), Drift-Resilient TabPFN (Helli et al., 2024), AdapTable (Kim et al., 2024), FTTA); (iii) temporal embeddings

Table 4
Key hyperparameters and their typical search ranges. "Reg" and "Cls" denote regression and classification settings, respectively. Bounds are not shared across searches and are given per search where they differ: "7-prior" is the main ensemble search behind Table 7, "adaptive" the adaptive-mechanism searches of Table 13, "cls tree-leaf" the tree-leaf classification search; unqualified ranges apply to all. This difference in search spaces is itself one reason columns from different searches are not directly comparable (Section 5.3). Every tuned value reported in Appendix D for these parameters lies within the bounds given here.

Parameter	Description	Search Range
w_i	Ensemble prior weight	$[0, 1]$, $\sum_i w_i = 1$
s_1	L1 correction scale	0.1–2.0 (7-prior), 0.3–1.2 (adaptive), 0.0–2.0 (cls)
k	Number of nearest neighbors	10–200 (7-prior), 20–200 (adaptive; LimiX cls), 5–100 (cls tree-leaf)
T_S	Short buffer temperature	0.5–20.0; 0.5–50.0 (cls tree-leaf, log scale)
T_L	Long buffer temperature	0.5–20.0; 1.0–12.0 (adaptive)
c	MAD clipping multiplier	1.0–4.0 (adaptive), 1.0–5.0 (7-prior), 1.0–10.0 (cls)
M_S	Short buffer capacity	128–1024 (reg), 50–2000 (cls, continuous)
Drift-aware blending (Eq. (12))		
ω_0	Base blend ratio	0.1–0.8
α	Drift sensitivity	0.5–10.0
W	Drift detection window	64–512
Embedding adaptation (Eq. (16))		
r	Low-rank dimension	{4, 8, 16}
η	Adaptation learning rate	10^{-5}–10^{-2}
λ	Weight decay	10^{-6}–10^{-2}
Cascaded L2 correction		
s_2	L2 correction scale	0.05–2.0
k_2	L2 neighbor count	27–290
T_2	L2 temperature	0.8–17.0

(Cai & Ye, 2025b); (iv) classical stream-learning methods (ARF Gomes et al., 2017, HAT Bifet & Gavaldà, 2009); and (v) simple online baselines (EWA, LLR).

On regression tasks, the PFC architecture achieves the best RMSE on four of five datasets, three of them by margins beyond run-to-run variation and the fourth (maps-routing) within it, as discussed below, with particularly strong margins on high-shift datasets: sberbank-housing (−2.5% vs. streaming TabPFN, the strongest baseline on that dataset) and weather (−2.7% vs. LightGBM). Temporal embeddings (Cai & Ye, 2025b) narrow the gap but still underperform on all shared datasets (on maps-routing, 0.1609 vs. 0.1606, by a margin within the noise band of the ties discussed below), suggesting that the full three-tier adaptation pipeline provides complementary robustness beyond static embeddings.

On classification tasks, the tree-plus-neural classification instantiation achieves competitive or state-of-the-art results: ecom-offers (0.6373, statistically tied with TARS FT-T at 0.6372) and homecredit-default (0.8697, numerically above the strongest static baseline, XGBoost at 0.8670). On homesite-insurance, the method ranks third (0.9652) behind two TARS variants. The ecom-offers margin (+0.0001 AUC) and the homesite-insurance gap (0.10 pp) are within run-to-run variation and are characterized as statistical ties rather than improvements; state-of-the-art claims are reserved for datasets with pronounced temporal shift. These characterizations have quantitative support. DeLong et al. (1988) 95% confidence intervals for the reported AUCs, computed from the per-sample test-stream predictions, are [0.9623, 0.9681] for homesite-insurance (SE 0.00146), [0.6302, 0.6444] for ecom-offers (SE 0.00360), and [0.8604, 0.8790] for homecredit-default (SE 0.00475). The differences to the strongest published competitors on the near-stationary datasets (−0.0010 to TARS TabM on homesite-insurance; +0.0001 to TARS FT-T on ecom-offers) amount to 0.69 and 0.03 standard errors respectively and are reported as statistical ties, as is the +0.0027 margin over XGBoost on homecredit-default (0.57 SE). By contrast, the +2.4 pp margin over the best end-to-end-retrained temporal baseline on homecredit-default (TARS TabM, 0.8456) corresponds to roughly five standard errors and is the basis of the state-of-the-art claim under pronounced shift. Interval widths are governed by the minority class (positives/negatives: 3,148/13,147 for homesite-insurance, 9,533/16,922 for ecom-offers, and 1,229/54,772 for homecredit-default): homecredit-default has the largest test stream (56,001 samples) but only 1,229 positives, hence the widest interval. A paired test on the correlated difference between two classifiers evaluated on the same stream would be sharper still, but requires the competing methods' per-sample scores, which are not published. On homesite-insurance, the proposed method's 0.9652 falls slightly below TARS TabM (0.9662) and TARS FT-T (0.9660), a gap of 0.10 percentage points from the best. This outcome is consistent with the architecture's design trade-off: homesite-insurance exhibits the least temporal shift among TabReD datasets, where feature distributions remain largely stationary across time segments, so the fast-tier correction and broader online adaptation mechanism provide limited lift. In this near-stationary regime, TARS's learned temporal routing, jointly optimized end-to-end with the backbone, can extract marginal gains from subtle distributional cues that post-hoc adaptation cannot capture. Conversely, on datasets with pronounced drift (sberbank-housing, weather, homecredit-default), the full architecture yields substantially larger improvements over all static approaches including TARS.

Online adaptation and the prequential protocol. The proposed method operates under the prequential protocol (Dawid, 1984): predict first, then update episodic memory with delayed ground-truth labels. Methods in the comparison tables span three categories: static (TabReD baselines, TARS, temporal embeddings), unsupervised TTA (Drift-Resilient TabPFN Helli et al., 2024, AdapTable Kim et al., 2024, FTAT Zhou et al., 2025), and online supervised (ARF (Gomes et al., 2017), HAT (Bifet & Gavaldà, 2009), EWA, LLR, Online XGB (Retrain), Online MLP (Fine-tune), Online FT-Transformer (Fine-tune), Online TabM (Fine-tune), and the proposed method).

The fast-tier kNN module contributes 2.3–9.1% RMSE reduction on regression tasks within the full architecture (Table 7, Ens. to +kNN columns). Classical stream-learning methods (ARF, HAT) perform poorly on TabReD (e.g., ARF: 0.5205 RMSE on cooking-time vs. the proposed method's 0.4737; 0.7954 AUC on homesite vs. the proposed method's 0.9652), confirming that they scale poorly to high-dimensional tabular embeddings. Simple online baselines (EWA, LLR) also underperform substantially, demonstrating that the proposed online adaptation mechanism is more effective at exploiting temporal feedback.

Comparison with online supervised baselines. To establish whether the proposed method's gains stem merely from exploiting delayed labels, rather than from the specific architecture, comparison is made against four strong online supervised baselines that also consume ground-truth labels as they arrive: (i) *Online XGB (Retrain)*, which periodically retrains XGBoost on all accumulated test-time labels (cumulative retraining every 2000–5000 samples); (ii) *Online MLP (Fine-tune)*, which incrementally fine-tunes a pretrained MLP via SGD on each revealed label batch; (iii) *Online FT-Transformer (Fine-tune)*; and (iv) *Online TabM (Fine-tune)*. All baselines have full access to the same label stream as the proposed method. On regression tasks (Table 5), the proposed method outperforms all online neural fine-tuning baselines on four of five regression datasets, with particularly large margins on weather (1.4237 vs. 1.5023 for Online FT-Transformer) and sberbank-housing (0.2173 vs. 0.2361 for Online FT-Transformer). On delivery-eta, Online FT-Transformer achieves a marginally lower RMSE (0.5441 vs. 0.5443), a gap of 0.04%

Table 5
Regression comparison (RMSE↓). Baselines from Cai and Ye (2025b), Rubachev et al. (2025), Wang et al. (2026). Best in **bold**, second-best underlined.

Category	Method	CT↓	SH↓	DE↓	WE↓	MR↓
TabReD baselines	XGBoost (Chen & Guestrin, 2016)	0.4823	0.2419	0.5468	1.4671	0.1616
	CatBoost (Prokhorenkova et al., 2018)	0.4823	0.2482	0.5465	1.4688	0.1619
	LightGBM (Ke et al., 2017)	0.4826	0.2468	0.5468	1.4625	0.1618
	MLP	0.4820	0.2508	0.5504	1.5470	0.1622
	MLP-PLR (Gorishniy et al., 2022)	0.4812	0.2438	0.5527	1.5177	0.1616
	FT-Transformer (Gorishniy et al., 2021)	0.4820	0.2440	0.5542	1.5104	0.1625
	MLP-PLR ens. (Gorishniy et al., 2022)	0.4806	0.2381	0.5518	1.4953	0.1612
	TabR (Gorishniy et al., 2024b)	0.4815	0.2447	0.5494	1.5186	0.1620
	TabICL (Qu et al., 2025)	0.4882	0.3489	0.5566	1.6365	0.1682
Temporal methods	Drift-Resilient TabPFN (Helli et al., 2024)	0.4818	0.2281	0.5535	1.6181	0.1662
	Modulated MLP (Cai & Ye, 2025a)	0.4782	0.2329	0.5578	1.5124	0.1616
	Modulated TabM (Cai & Ye, 2025a)	0.4775	0.2280	0.5574	1.5194	0.1612
	Koodos (Cai et al., 2024)	0.4807	0.2359	0.5554	1.5524	0.1626
	FTTA MLP (Zhou et al., 2025)	0.4835	0.2488	0.5541	1.5600	0.1636
	FTTA FT-T (Zhou et al., 2025)	0.4795	0.2294	0.5469	1.5134	0.1621
	TARS MLP (Wang et al., 2026)	0.4770	0.2353	0.5473	1.5113	0.1612
	TARS TabM (Wang et al., 2026)	0.4777	0.2264	0.5476	1.4804	0.1608
	TARS FT-T (Wang et al., 2026)	0.4761	0.2290	0.5457	1.5039	0.1617
TE	MLP + TE (Cai & Ye, 2025b)	0.4801	0.2431	0.5518	1.5319	0.1621
	ModernNCA + TE (Cai & Ye, 2025b)	0.4791	0.2255	0.5535	1.4903	0.1617
	TabM + TE (Cai & Ye, 2025b)	0.4791	0.2321	0.5488	1.4812	0.1609
Online	ARF (Gomes et al., 2017)	0.5205	0.2499	0.6438	1.9013	0.1991
	HAT (Bifet & Gavaldà, 2009)	0.5299	0.2923	0.5941	2.1645	0.2078
	EWA	0.4876	0.2596	0.5611	1.7488	0.1675
	LLR	0.5039	0.3308	0.5914	1.7836	0.1982
	Online XGB (Retrain) (Chen & Guestrin, 2016)	0.4820	0.2550	0.5459	1.5474	0.1629
	Online MLP (Fine-tune)	0.4807	0.2492	0.5461	1.5279	0.1644
	Online FT-Transformer (Fine-tune) (Gorishniy et al., 2021)	0.4796	0.2361	**0.5441**	1.5023	0.1620
	Online TabM (Fine-tune) (Gorishniy et al., 2024a)	0.4837	0.2491	0.5473	1.5552	0.1740
	TTLT (Kang & Kang, 2026)	0.4834	0.2754	0.5541	1.5573	0.1673
	Streaming TabPFN (Lourenço et al., 2025)	0.4813	0.2228	0.5515	1.5275	0.1700
	Ours	**0.4737**	**0.2173**	0.5443	**1.4237**	**0.1606**

that falls within statistical noise; delivery-eta exhibits gradual drift, a regime in which fine-tuning can match the proposed post-hoc adaptation pipeline. The same reading applies to a margin in the method's favor: the maps-routing advantage over TARS TabM in Table 5 (0.1606 vs. 0.1608) is of the same absolute size, so that entry is likewise best read as a tie rather than a clear improvement, and the regression state-of-the-art claim rests on the three datasets with larger margins (cooking-time, sberbank-housing, and weather). On classification tasks (Table 6), the proposed method clearly dominates Online FT-Transformer and Online TabM on all three datasets (e.g., homesite-insurance: 0.9652 vs. 0.9405 and 0.9334; ecom-offers: 0.6373 vs. 0.5967 and 0.6090; homecredit-default: 0.8697 vs. 0.8440 and 0.8321). These results confirm that the proposed method's strong performance reflects the effectiveness of the full three-tier adaptation strategy, where frozen priors, lightweight reconfiguration, and non-parametric fast-tier correction work together to deliver stronger adaptation than gradient-based online retraining or fine-tuning, without updating the frozen prior parameters.

Comparison with retrieval-based and foundation-model adaptation. Three further baselines isolate the two design choices most closely related to the proposed fast tier. *TTLT* (Kang & Kang, 2026) replaces the gradient-free episodic correction with gradient-based per-query local fine-tuning over training-set neighbors; *streaming TabPFN* (Lourenço et al., 2025) replaces the dual-buffer episodic memory with a single sliding context window prompted into a frozen foundation model; and *TabICL* (Qu et al., 2025) is a more recent tabular foundation model evaluated as a static prior. For a like-for-like comparison, all three use the full feature set (numerical, binary, and categorical): the TTLT backbone matches the published MLP pipeline with one-hot-encoded categoricals, streaming TabPFN receives the categorical columns through TabPFN's native categorical-feature handling, and TabICL ingests them as additional input columns. The proposed method outperforms all three on all eight datasets. Streaming TabPFN is the strongest of the three, narrowly trailing on its strongest datasets (sberbank-housing 0.2228 vs. 0.2173; cooking-time 0.4813 vs. 0.4737, where it already surpasses the gradient-boosted trees), which confirms that a frozen foundation model prompted with recent labels is a competitive training-free adapter, yet the dual-buffer episodic memory with drift-aware blending consistently extracts further gains, and the advantage widens on wide, heavily categorical datasets where the foundation model degrades (homecredit-default 0.7718 vs. 0.8697). TTLT tracks its MLP backbone and is beaten across the board (e.g., homesite-insurance 0.9473 vs. 0.9652; weather 1.5573 vs. 1.4237) while incurring 40–47 ms of gradient computation *per test sample*, three orders of magnitude above the proposed gradient-free correction. TabICL, evaluated as a static prior under its default preprocessing and despite being newer than the LimiX prior used here, does not surpass either the proposed method or the gradient-boosted baselines on these temporally shifted benchmarks (e.g., weather 1.6365; ecom-offers 0.5732), with its largest shortfalls on the highest-shift regression tasks; this is consistent with the known tendency of tabular foundation models to trail gradient-boosted trees under distribution shift, and reinforces that a strong static prior alone does not resolve temporal distribution shift without an explicit online-adaptation mechanism.

The aggregate comparison in Fig. 6 highlights the same pattern visually: the proposed method remains best or competitive across both regression and classification settings. Unsupervised TTA methods (Drift-Resilient TabPFN, AdapTable) underperform the static baselines on most datasets; the exception is Drift-Resilient TabPFN on sberbank-housing (0.2281) and ecom-offers (0.6163), where it leads the standard tabular baselines, though it remains behind the best temporal methods on both;

Table 6
Comparison with published methods on classification datasets (AUC↑). Best in **bold**, second-best underlined. DeLong 95% confidence intervals for the proposed method: ±0.0029 (HI), ±0.0071 (EO), ±0.0093 (HD); see Section 5.2.

Category	Method	HI↑	EO↑	HD↑
TabReD baselines	XGBoost (Chen & Guestrin, 2016)	0.9601	0.5763	0.8670
	CatBoost (Prokhorenkova et al., 2018)	0.9606	0.5596	0.8621
	LightGBM (Ke et al., 2017)	0.9603	0.5758	0.8664
	MLP	0.9500	0.6015	0.8545
	MLP-PLR (Gorishniy et al., 2022)	0.9621	0.5957	0.8568
	FT-Transformer (Gorishniy et al., 2021)	0.9622	0.5775	0.8571
	MLP-PLR ens. (Gorishniy et al., 2022)	0.9629	0.5981	0.8585
	TabR (Gorishniy et al., 2024b)	0.9503	0.6019	0.8557
	TabPFN (Hollmann et al., 2025)	0.9490	0.5952	0.7708
	TabICL (Qu et al., 2025)	0.9435	0.5732	0.8147
Temporal methods	Drift-Resilient TabPFN (Helli et al., 2024)	0.9558	0.6163	0.8144
	AdapTable (Kim et al., 2024)	0.8598	0.5832	0.5367
	Modulated MLP (Cai & Ye, 2025a)	0.9560	0.6257	0.5528
	Modulated TabM (Cai & Ye, 2025a)	0.9626	0.6362	0.8437
	Koodos (Cai et al., 2024)	0.9527	0.6292	0.5448
	FTTA MLP (Zhou et al., 2025)	0.9441	0.6295	0.5529
	FTTA FT-T (Zhou et al., 2025)	0.9624	0.6255	0.6056
	TARS MLP (Wang et al., 2026)	0.9564	0.6283	0.5567
	TARS TabM (Wang et al., 2026)	**0.9662**	0.6358	0.8456
	TARS FT-T (Wang et al., 2026)	0.9660	0.6372	0.5952
TE	MLP + TE (Cai & Ye, 2025b)	0.9471	0.6252	0.5519
	ModernNCA + TE (Cai & Ye, 2025b)	0.9620	0.6356	0.8316
	TabM + TE (Cai & Ye, 2025b)	0.9629	0.6271	0.8363
Online	ARF (Gomes et al., 2017)	0.7954	0.5985	0.7101
	HAT (Bifet & Gavaldà, 2009)	0.7209	0.5505	0.6730
	EWA	0.8367	0.5538	0.7744
	LLR	0.8864	0.6055	0.7986
	Online XGB (Retrain) (Chen & Guestrin, 2016)	0.9383	0.6015	0.8513
	Online MLP (Fine-tune)	0.9285	0.6108	0.8110
	Online FT-Transformer (Fine-tune) (Gorishniy et al., 2021)	0.9405	0.5967	0.8440
	Online TabM (Fine-tune) (Gorishniy et al., 2024a)	0.9334	0.6090	0.8321
	TTLT (Kang & Kang, 2026)	0.9473	0.5943	0.8421
	Streaming TabPFN (Lourenço et al., 2025)	0.9540	0.6234	0.7718
	Ours	0.9652	**0.6373**	**0.8697**

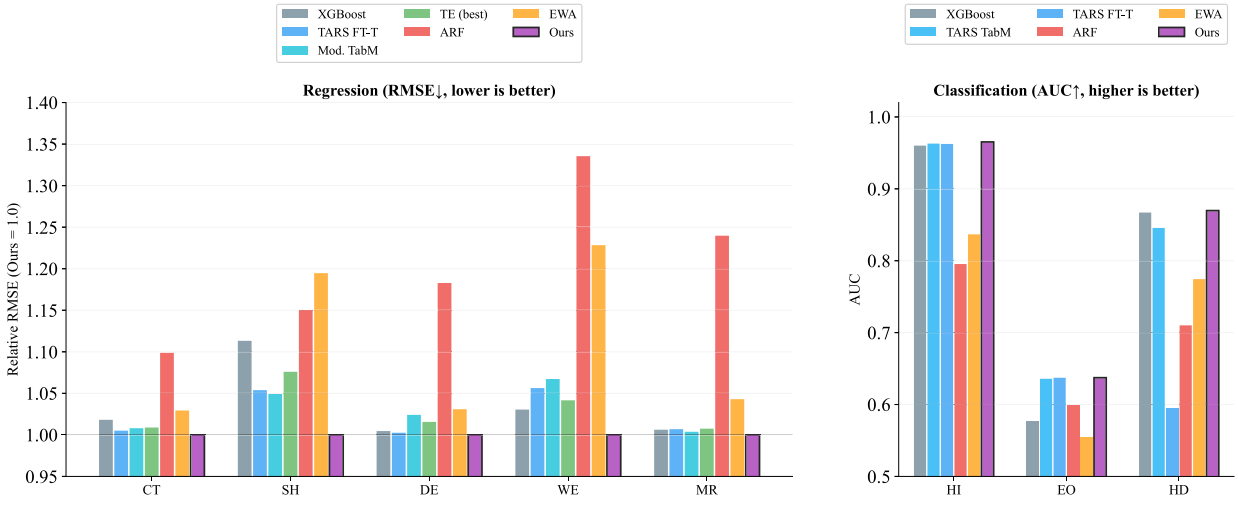

Fig. 6. Left: Relative RMSE (normalized, lower is better). **Right:** Classification AUC (higher is better). Proposed method achieves best or competitive results across all datasets.

overall these results confirm that label-free adaptation alone is insufficient for temporal shifts in tabular data.

The architecture maintains a unified three-tier structure across task types, with task-specific slow tier instantiation (LimiX for regression, tree ensemble for classification). The embedding choice for classification kNN is analyzed in Section 5.3.3.

5.3. Ablation studies

This section systematically evaluates the contribution of each architectural component through controlled ablations. A note on cross-table comparability: each ablation isolates a different mechanism and therefore runs the pipeline under a different controlled configuration, so the

Table 7
Tier progression ablation. Prior: single model. Ens.: weighted ensemble. +kNN: online correction. Full: all mechanisms. Best in **bold**. On maps-routing the first three columns are computed over the LimiX prior pool, which PCA reduction degrades on this dataset (Section 5); the Full column reports the deployed configuration, which replaces that pool with a tree-plus-neural one whose ensemble-only RMSE is 0.1615, so the fast tier contributes 0.1615 → 0.1606 there. Every other regression row uses one prior pool throughout. On the classification rows, Prior, Ens., and +kNN are computed on the tree-ensemble configuration, while Full reports the deployed per-dataset pool of Section 5: on ecom-offers and homesite-insurance this blends neural priors into the ensemble, and on homecredit-default it coincides with the tree ensemble.

Regression (RMSE↓)				
Dataset	Prior	Ens.	+kNN	Full
Cooking-time	0.4916	0.4890	0.4758	**0.4737**
Sberbank-housing	0.2651	0.2435	0.2218	**0.2173**
Delivery-eta	0.5821	0.5602	**0.5443**	**0.5443**
Weather	1.6146	1.5859	1.4423	**1.4237**
Maps-routing	0.1723	0.1726	0.1686	**0.1606**
Classification (AUC↑)				
Dataset	Prior	Ens.	+kNN	Full
Homesite-insurance	0.9601	0.9640	0.9649	**0.9652**
Ecom-offers	0.5763	0.6233	0.6280	**0.6373**
Homecredit-default	0.8670	0.8684	0.8632	**0.8697**

same dataset can appear with slightly different numbers in different tables. Table 7 reports the deployed operating point and is the reference; Tables 8–10 toggle the cascaded L2 layer under the stripped-down or full-adaptive regimes stated in their captions; Table 12 varies only the embedding space on top of the tree ensemble; and Table 13 compares the adaptive mechanisms, with classification instantiated on LimiX priors rather than the deployed tree ensemble. Each caption states the exact configuration and how it relates to Table 7. A second, less visible source of incomparability is that these searches do not share hyperparameter bounds: the bounds are listed per search in Table 4 and differ between them, so a column can be optimal within its own search space and still differ from Table 7 by more than the mechanism it isolates. The same caveat applies to the supporting analyses in Table 11 and Appendices A and B, which were produced by separate analysis runs with their own prior caches and are therefore not numerically identical to Table 7 either; where a number is used to support a claim in the main text, it is taken from Table 7.

5.3.1. Component contribution analysis

Table 7 presents the progressive improvement as each architectural tier is added, providing a unified view of how the system builds from a single frozen prior to the full adaptive architecture.

The results reveal several key insights. First, the full system achieves consistent improvements over the single-prior baseline across all datasets, with regression RMSE reductions ranging from 3.6% (cooking-time) to 18.0% (sberbank-housing). Second, each tier contributes progressively but with varying magnitudes: ensemble blending in the intermediate tier provides modest gains of 0.5–8.1% over the single prior on most regression datasets, while kNN correction in the fast tier adds substantial improvement of 2.3–9.1% over the ensemble-only baseline. Third, the full system with cascaded L2 and adaptive mechanisms achieves the best result on 4 of 5 regression datasets; delivery-eta is an exception where the +kNN configuration already reaches optimal performance, and on maps-routing the Full column additionally reflects the change of prior pool discussed next. Fourth, maps-routing exhibits a distinctive pattern that illustrates the modularity of the slow tier. The LimiX prior (0.1723) underperforms XGBoost (0.1616 in Table 5) because the 984 raw features must be PCA-reduced before cache generation, and blending over the weakened LimiX variants slightly degrades performance (0.1726). Online correction recovers most of that deficit within the same pool (0.1686) but does not close it, which is the expected behavior: an additive residual correction cannot repair information the priors never received. The deployed configuration therefore replaces the pool itself with a tree-plus-neural one (ensemble-only RMSE 0.1615), on top of which the unchanged fast tier reaches 0.1606. The point is not that online adaptation compensates for a poor prior pool, which it does only in part, but that the frozen slow tier is interchangeable: the same correction mechanism operates without modification over a different prior family.

For classification, the progression is similarly consistent: the full system achieves AUC of 0.9652 (homesite), 0.6373 (ecom), and 0.8697 (homecredit). Notably, ensemble blending contributes larger relative gains on classification tasks (e.g., +4.7 pp AUC (+8.2% relative) on ecom-offers from Prior to Ens.), reflecting the greater diversity among tree-based priors compared to the highly correlated LimiX variants used for regression.

Quantifying the kNN contribution from the Ens. to +kNN columns in Table 7, the online correction reduces RMSE by 2.7% (cooking-time), 8.9% (sberbank-housing), 2.8% (delivery-eta), 9.1% (weather), and 2.3% (maps-routing). The maps-routing figure is measured within the LimiX pool of the first three columns; within the deployed tree-plus-neural pool the fast tier contributes 0.1615 → 0.1606, i.e. 0.6%. The largest gains occur on datasets with pronounced temporal shift (sberbank-housing, weather), confirming that online non-parametric adaptation captures shift patterns that frozen priors cannot, with benefit scaling with shift severity. Figs. 7–9 visualize this effect from complementary perspectives: temporal error trajectories, cumulative component gains, and per-sample correction behavior.

Cascaded L2: regression vs. classification. We characterize the second-pass correction in two regimes. In a *no-adaptation* regime (L1 held fixed at its +kNN configuration, a static L2 added on top, and the drift-aware blending, online embedding adaptation, and L2 gate all switched off), the raw effect of L2 is small and dataset-dependent (Table 8): it reduces RMSE on cooking-time (−0.0021) and sberbank-housing (−0.0011) but slightly increases it on delivery-eta (+0.0015) and weather (+0.0019), where the post-L1 residuals are close to white noise and a second non-parametric pass mainly fits sampling noise. In the *full adaptive* system, however, we re-run end to end and toggle only L2 on the two datasets whose caches reconstruct faithfully (Table 9): there L2 is consistently, if mildly, beneficial (cooking-time −0.0018 and weather −0.0007), and the adaptive gate never fires, because L2 lowers error and is therefore correctly retained. Weather is instructive: it appears as a small regression in the no-adaptation regime but becomes a small improvement once the full adaptive base is in place, indicating that the apparent harm is an artifact of the stripped-down setting rather than a property of the deployed system. We therefore position the L2 gate as a *safety backstop*: it guarantees that the second pass cannot degrade accuracy (by permanently disabling itself whenever its running contribution to error turns negative), yet on every run we could reproduce end to end it did not need to fire. Consistent with this, the Full column of Table 7 is never worse than the +kNN column on any regression dataset.

For classification the gains are likewise marginal, but for a different reason. On homesite-insurance and ecom-offers, L2 contributes at most 0.03 pp AUC, consistent with uniformly distributed classification errors after ensemble optimization; a direct on/off ablation confirms $|\Delta_{L2}| \leq 0.03$ pp on all three classification datasets (Table 10). Homecredit-default is a more instructive case. In the tier-progression configuration of Table 7 the first kNN pass lowers AUC (0.8684 → 0.8632), whereas with the per-dataset tuned retrieval embeddings of Table 12 the same correction *improves* it (to 0.8695–0.8700), and with a poorly matched embedding it collapses (0.7905 under TabPFN embeddings). What this dataset demonstrates is therefore not that online correction is harmful at high

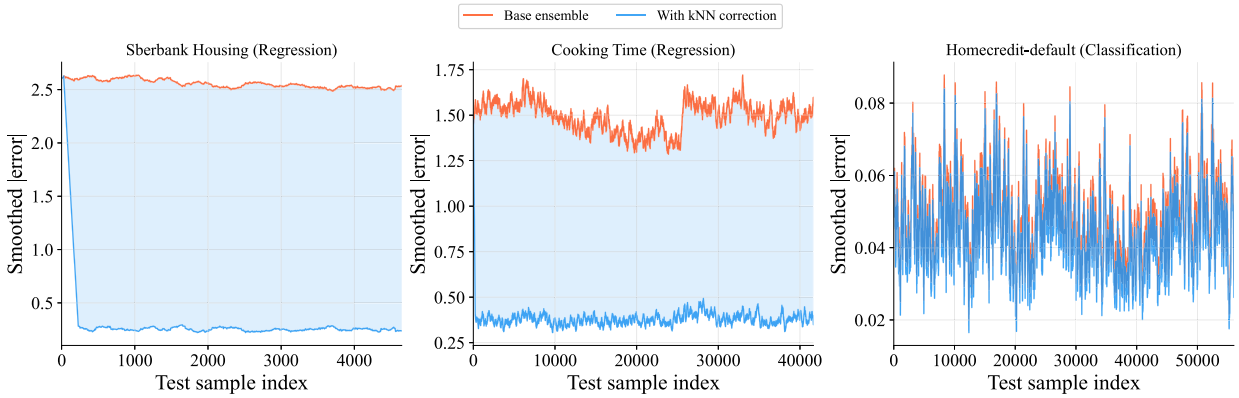

Fig. 7. Temporal evolution of prediction error with and without kNN correction (200-sample smoothing). Shaded area shows error reduction from kNN correction.

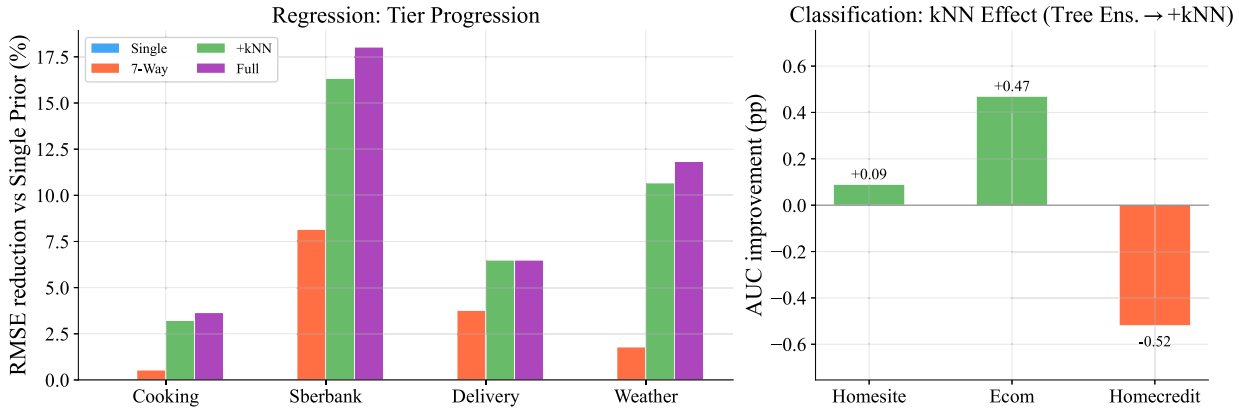

Fig. 8. Left: Regression tier progression (cumulative RMSE reduction). **Right:** Classification AUC change from the Ens. to the +kNN column of Table 7; the homecredit-default decrease in this configuration reflects the retrieval-embedding dependence discussed in the text.

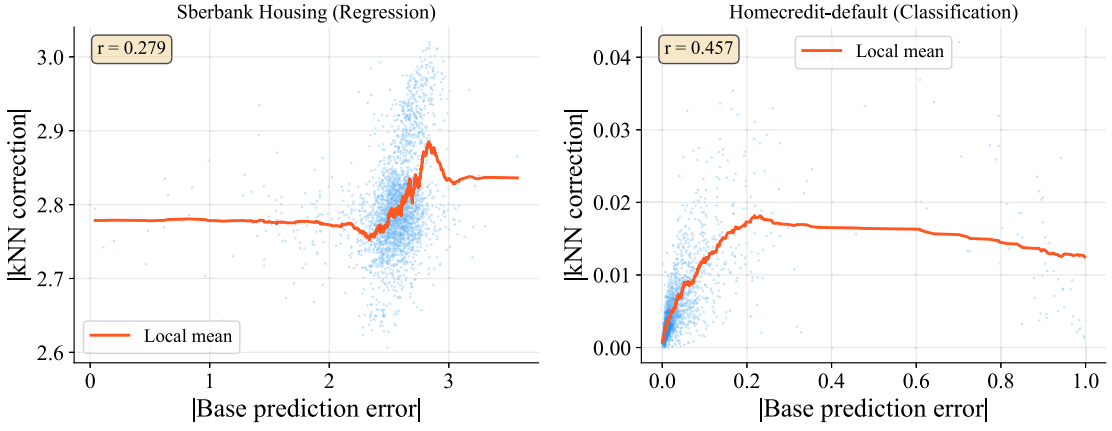

Fig. 9. Per-sample kNN correction magnitude vs. base prediction error. The local-mean trend (orange) grows with base error, and the per-sample correlation is positive but moderate ($r = 0.279$ for Sberbank-housing regression and $r = 0.457$ for Homecredit-default classification), indicating that the correction is larger where the ensemble errs more.

dimension, but that with 614 features the outcome is governed by the space in which neighbors are retrieved: the spread across embeddings (0.79 to 0.87 AUC) is an order of magnitude larger than on the other two classification datasets. The deployed configuration, which selects the retrieval embedding per dataset, reaches 0.8697; the isolated contribution of the cascaded L2 layer here is −0.004 pp (Table 10), so the recovery is attributable to the choice of retrieval embedding rather than to the second pass. Overall, L2 is best understood as a safety-backstop refinement: mildly beneficial where residual structure remains and prevented from harming accuracy by the gate, rather than a consistent primary source of accuracy.

5.3.2. *Ensemble diversity analysis*

Table 11 examines the role of architectural heterogeneity in ensemble performance, focusing on the contribution of XGBoost as a structurally distinct prior alongside the six LimiX variants.

Several patterns emerge from the diversity analysis in Table 11. First, XGBoost contributes 21–48% of total ensemble diversity despite being only one of seven priors, confirming that architectural heterogeneity (tree-based vs. transformer-based) generates more diversity than hyperparameter variation within a single architecture. Second, the XGBoost–LimiX correlation varies substantially across datasets (0.658–0.981), with lower correlation on classification tasks where feature interactions

Table 8
Isolated effect of the cascaded L2 layer in the *no-adaptation* regime (regression, RMSE↓). L1 is fixed at the +kNN configuration of Table 7 and a static L2 is added on top, with drift-aware blending, embedding adaptation, and the L2 gate disabled, so Δ_{L2} measures the raw L2 contribution. L2 helps on two datasets and is mildly detrimental on two; under the full adaptive system this residual effect vanishes (Table 9).

Dataset	L1 (+kNN)	+L2 (static)	Δ_{L2}
Cooking-time	0.4758	**0.4737**	−0.0021
Sberbank-housing	0.2218	**0.2207**	−0.0011
Delivery-eta	**0.5443**	0.5458	+0.0015
Weather	**1.4423**	1.4442	+0.0019

Table 9
Cascaded-L2 ablation within the full adaptive system (regression, RMSE↓). The complete pipeline (ensemble + dual-buffer kNN + drift-aware blending + adaptive embedding) is run end-to-end and only the L2 layer is toggled: "L2 off" ($s_2 = 0$), "L2 on" (adaptive gate retained), and "L2 raw" (gate disabled). On both datasets the gate retains L2 and the layer is mildly beneficial; weather in particular *improves*, unlike its no-adaptation estimate (+0.0019) in Table 8. Results are reported for cooking-time and weather, the two datasets for which the deployed pipeline is reproduced end-to-end. Because this reproduction regenerates the prior caches, its absolute numbers differ marginally from the deployed run in Table 7 (e.g. cooking-time 0.4739 vs. 0.4737); the L2 on/off contrast within this table is computed under identical conditions.

Dataset	L2 off	L2 on (gate)	L2 raw (no gate)	Δ_{L2}
Cooking-time	0.4757	**0.4739**	0.4739	−0.0018
Weather	1.4243	**1.4236**	1.4236	−0.0007

Table 10
Cascaded-L2 ablation on classification (AUC↑). L1 is the tree-ensemble + dual-buffer kNN configuration and "+L2 (raw)" adds the second cascaded kNN pass with the gate disabled, so Δ_{L2} is the raw layer effect. The second pass changes AUC by at most 0.03 percentage points on every dataset, indicating that after ensemble optimization the classification residuals carry little further structure for L2 to exploit. The L1 baseline here excludes the remaining adaptive mechanisms, so its values may differ from the Full column of Table 7 (e.g. homesite 0.9643 vs. 0.9652).

Dataset	L1 (off)	+L2 (raw)	Δ_{L2} (pp)
Homesite-insurance	0.9643	0.9643	+0.001
Ecom-offers	0.6294	0.6291	−0.026
Homecredit-default	0.8697	0.8697	−0.004

differ more between model families. Third, the dataset with the largest ensemble gain is also among the two most diverse (sberbank-housing, 5.6% diversity, +8.1% RMSE reduction from Prior to Ens. in Table 7; weather has the highest ratio at 5.8%), which is broadly consistent with the Krogh–Vedelsby decomposition (Section 4.2); the correspondence is not monotone across the remaining datasets, however, with delivery-eta gaining 3.8% at the lowest diversity ratio (2.1%) and maps-routing showing no ensemble gain at 3.9% diversity.

5.3.3. Embedding space analysis

The choice of embedding space for kNN correction is critical, particularly for classification where foundation model embeddings may not align with discriminative boundaries. Table 12 compares different embedding types for the kNN step applied to the tree ensemble.

Figs. 10 and 11 summarize the classification embedding analysis. Tree leaf embeddings match or exceed the LimiX foundation-model embedding for classification kNN, with a clear advantage only on ecom-

Table 11
Ensemble diversity analysis. XGB–LimiX ρ: prediction correlation. XGB Wt.: Optuna-assigned weight. 6→7 Δ: relative change in the task error metric (RMSE for regression, 1 − AUC for classification) from adding XGBoost; negative values indicate improvement. Div. Ratio: Krogh–Vedelsby ambiguity. XGB Div. Share: XGBoost's diversity contribution. All quantities are computed on the six-LimiX-plus-XGBoost prior pool used for the diversity analysis in this section; several datasets deploy a different pool for their main results (Section 5). For the binary-classification datasets the Krogh–Vedelsby decomposition is in squared-error units and is therefore not comparable with the AUC reported elsewhere, so the last three columns are omitted. For maps-routing the six-prior reference run required by the 6→7 comparison is not available, so the first three columns are omitted. Best in **bold**.

Dataset	XGB–LimiX ρ	XGB Wt.	6→7 Δ	Div. Ratio	XGB Div. Share	Ens. RMSE
Ecom-offers	0.658	27.3%	−0.17%	N/A	N/A	N/A
Homesite-ins.	0.812	33.4%	−2.43%	N/A	N/A	N/A
Sberbank	0.937	15.0%	−0.26%	5.6%	31.9%	0.2393
Delivery	0.960	47.9%	−1.02%	2.1%	24.3%	0.5627
Cooking	0.969	44.6%	−0.43%	2.5%	21.2%	0.4878
Weather	0.981	27.6%	−1.26%	5.8%	30.2%	1.5786
Maps	N/A	N/A	N/A	3.9%	47.8%	0.1726

Table 12
Classification kNN embedding ablation (AUC↑). Tree Ens.: tree ensemble without kNN. +columns: tree ensemble with kNN using different embedding spaces. Prod.: full-model configuration with optimized embedding choice per dataset, shown in **bold**. On homecredit-default, TabPFN embedding kNN substantially underperforms the tree-ensemble baseline and is omitted from the figure for readability. The +embedding columns apply the kNN correction alone on top of the tree ensemble; only the Prod. column includes the full adaptive configuration and therefore coincides with the Full column of Table 7. On homecredit-default the +XGB leaf column is marginally higher than Prod. (0.8700 vs. 0.8697), a difference well inside the DeLong standard error of Section 5.2.

Dataset	Tree Ens.	+LimiX	+XGB leaf	+CatBoost leaf	+TabPFN	Prod.
Homesite	0.9640	0.9641	0.9642	0.9642	0.9652	**0.9652**
Homecredit	0.8684	0.8698	0.8700	0.8695	0.7905	**0.8697**
Ecom	0.6233	0.6260	0.6282	0.6295	0.6282	**0.6373**

offers (+0.0022 to +0.0035 AUC); on homesite-insurance and homecredit-default the differences (+0.0001 to +0.0002, and −0.0003 for CatBoost leaves on homecredit-default) lie well inside the DeLong standard errors of Section 5.2 and are not read as improvements. This advantage arises because tree leaf indices naturally encode decision boundaries that are directly relevant to the classification task, whereas foundation model embeddings are optimized for reconstruction rather than discrimination. Interestingly, no single embedding type dominates across all datasets: TabPFN excels on homesite-insurance, XGBoost leaves on homecredit-default, and CatBoost leaves on ecom-offers. This motivates the per-dataset embedding selection in the full-model configuration. Notably, although TabPFN embeddings degrade the tree ensemble on homecredit-default (614 features), they still improve over TabPFN's own standalone prediction (0.7708 → 0.7905 AUC), so the kNN correction still improves on the prior it is applied to even under a poorly matched embedding, although the resulting AUC then remains far below the tree-ensemble baseline: the mechanism is robust in direction but not in magnitude.

5.3.4. Adaptive mechanism analysis

The drift-aware buffer blending and online embedding adaptation mechanisms (Section 4.3.3) are designed to handle non-stationary distributions. Table 13 evaluates their individual and combined contributions.

The adaptive mechanisms summarized in Table 13 provide complementary benefits, and Fig. 12 illustrates the corresponding temporal behavior of the drift-aware blend ratio. Drift-aware buffer management (AdapBuf) shows column-level improvements over the fixed-ω base-

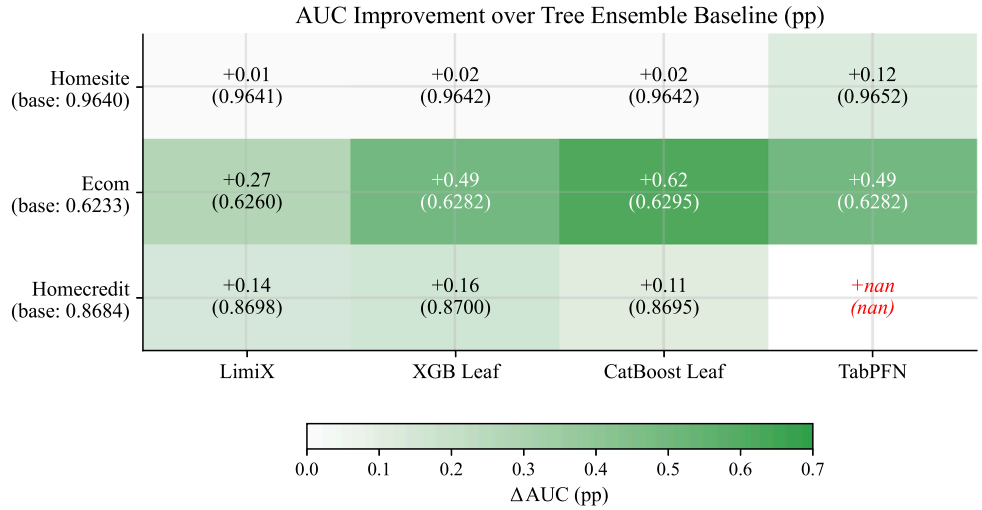

Fig. 10. AUC improvement (percentage points) over tree ensemble baseline by embedding type; cell shading encodes the magnitude of the improvement.

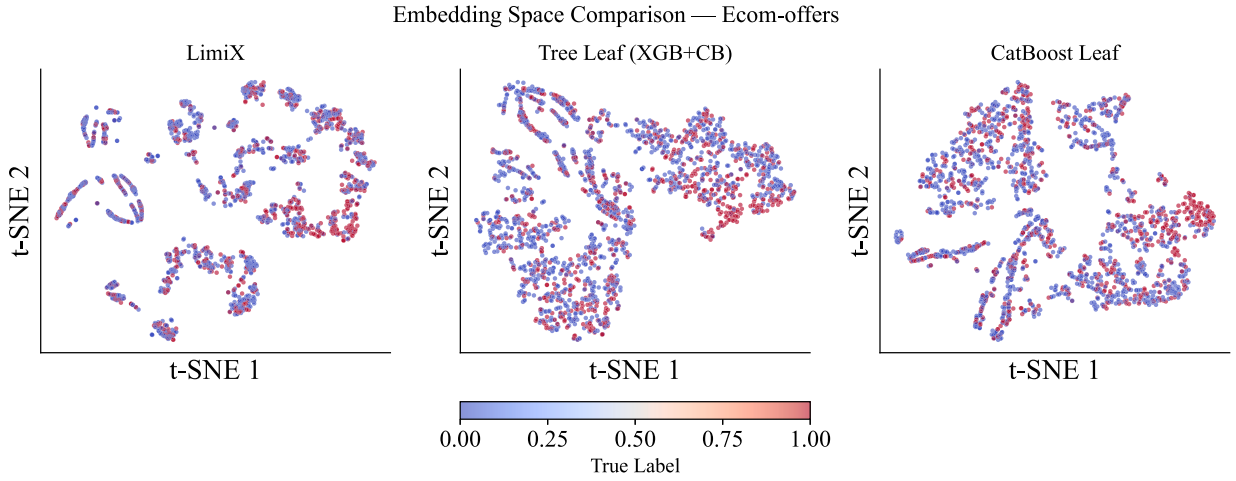

Fig. 11. t-SNE visualization of embedding spaces for ecom-offers (2000 test points). Tree leaf embeddings produce more discriminative structures than foundation model embeddings.

Table 13
Adaptive mechanism ablation. Fixed-ω: static buffer blend ratio. AdapBuf: drift-aware ω_t (Eq. (12)). AdapEmb: online low-rank embedding adaptation. Combined: both mechanisms active. Classification uses LimiX embeddings. Best in **bold**. The regression Fixed-ω column corresponds to the +kNN configuration of Table 7; each adaptive column is evaluated at its own tuned operating point, and the adaptive searches do not share hyperparameter bounds with the main search of Table 7, so column differences reflect the mechanism together with the accompanying re-tuning of the shared hyperparameters under a different search space. The classification rows use the LimiX-based instantiation for controlled comparison and are therefore not comparable to the tree-ensemble numbers of Table 7.

Regression (RMSE↓)				
Dataset	Fixed-ω	AdapBuf	AdapEmb	Combined
Cooking-time	0.4758	0.4748	**0.4743**	0.4745
Sberbank-housing	0.2218	0.2191	0.2183	**0.2179**
Delivery-eta	**0.5443**	0.5452	0.5452	0.5453
Weather	1.4423	1.4340	1.4347	**1.4290**
Maps-routing	0.1686	**0.1683**	**0.1683**	0.1689
Classification (AUC↑, LimiX-based instantiation)				
Homesite-insurance	0.9236	0.9279	0.9300	**0.9308**
Ecom-offers	0.6179	0.6236	0.6244	**0.6259**
Homecredit-default	0.8002	0.8052	0.8107	**0.8229**

line on 7 of 8 datasets, with the largest column gaps on the high-shift datasets sberbank-housing (−1.2% RMSE) and weather (−0.6% RMSE); as for AdapEmb, these gaps include the re-tuning of the shared hyperparameters. A paired toggle at the frozen deployed operating point (Appendix E), deterministic and free of seed and search variance, isolates the mechanism's causal contribution: on sberbank-housing it improves RMSE by 1.69×10^{-4} (0.077%), and the blend ratio is demonstrably active on every dataset examined (Fig. 12). The AdapEmb column shows comparable numbers, but most of its gap to Fixed-ω reflects the re-tuning of the shared hyperparameters at each operating point rather than the mechanism itself (see the table caption): the paired study of Appendix E, which isolates the adapter's causal contribution with all other hyperparameters frozen, finds a statistically significant but substantially smaller effect, operating by adjusting which points are considered neighbors rather than how neighbor corrections are weighted. The combined configuration achieves the best result on 5 datasets (sberbank, weather, and all three classification tasks), consistent with the two mechanisms addressing complementary aspects of distribution shift at their respective tuned operating points.

Notably, the fixed-ω baseline remains optimal on delivery-eta, and on maps-routing the adaptive margin is negligible (0.1686 vs. 0.1683); both are datasets with gradual or minimal temporal shift. On these datasets, the drift signal ρ_t remains near unity throughout the test stream, and the adaptive mechanisms correctly default to conser-

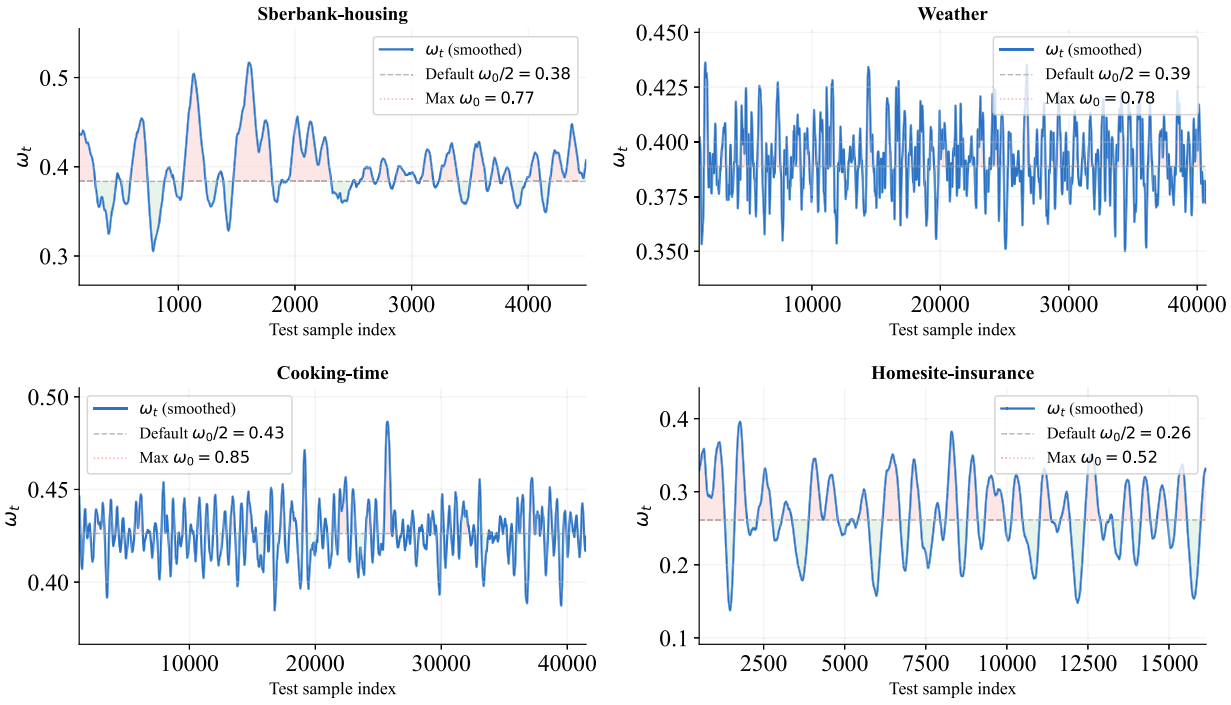

Fig. 12. Temporal evolution of drift-aware blend ratio ω_t (300-sample average). Red: recency bias; green: stability bias. High-shift datasets show sustained elevated ω_t.

vative behavior rather than introducing unnecessary variance. This self-regulating property, adapting aggressively when shift is detected and conservatively when it is not, validates the PFC-inspired context-switching formulation.

Note that classification results in Table 13 use the LimiX-based slow-tier instantiation for controlled comparison. The full classification system uses tree-ensemble priors with kNN in leaf embedding space (Table 7), which yields stronger baselines but follows the same adaptive mechanism design.

Failure cases and limitations. The analysis above also delineates where the architecture provides little or no benefit. On near-stationary datasets the fast tier adds limited lift: on homesite-insurance, which exhibits the least temporal shift, the method ranks behind two end-to-end-retrained TARS variants (Section 5.2); on delivery-eta and maps-routing the drift signal ρ_t stays near unity and the adaptive mechanisms correctly default to conservative behavior. When the prior pool is suboptimal (maps-routing, where PCA reduction degrades the LimiX priors), online correction recovers most of the gap within that pool (0.1726 → 0.1686) but does not close it; the effective remedy there is to replace the priors, as the deployed configuration does, rather than to correct harder. On high-dimensional data the outcome of the first kNN pass depends strongly on the retrieval embedding: on homecredit-default (614 features) it lowers AUC in the tier-progression configuration but improves it under the tuned per-dataset embeddings of Table 12, and degrades it severely under a mismatched one. The second corrective pass is likewise not universally beneficial: in the no-adaptation regime it slightly increases RMSE on delivery-eta and weather (Table 8), where post-L1 residuals are close to white noise. These cases are consistent with the self-pruning behavior of the design: when a component is unhelpful, its tuned strength collapses ($\eta \approx 0$ for the embedding adapter, the adaptive gate disabling the L2 correction, $\alpha \to 0$ for drift blending), so inactive components do not degrade performance. For the embedding adapter this self-pruning is not in tension with the activation evidence of Appendix E: the update of Eq. (17) carries weight decay on both factors, so where the neighbor gradient carries no usable signal the product $\boldsymbol{U}\boldsymbol{V}^\top$ decays back to zero and the transform returns to the identity, whereas the appendix reports operating points where the gradient does carry signal and the transform grows away from it. Both describe the same dynamics at different points of the hyperparameter space. Finally, the residual tuning burden concentrates on the correction scale s_1 (worst-case probe-stream degradation 1.75%, Section 5.3.5); this is the axis where a fresh deployment benefits most from the transferable defaults of Table 3.

Beyond the evaluated benchmarks, three regimes bound the method's expected applicability. Under *abrupt* concept drift, the drift statistic needs on the order of W samples to respond and the short buffer needs M_S post-shift labels to refill, so for a transition period the correction is computed from pre-shift residuals; the MAD clipping and the convex blend bound the resulting damage (Proposition 1), but the method reacts to change points rather than anticipating them. Under *severe* distributional change that invalidates the frozen prior's representation itself, rather than merely biasing its outputs, an additive residual correction in a fixed embedding geometry can compensate only partially, as the maps-routing case already illustrates in miniature; when the embedding manifold no longer organizes the new distribution, retraining or replacing the prior becomes unavoidable. Finally, because the fast tier consumes revealed labels under the prequential protocol, streaming environments with *delayed or scarce labels* weaken the correction proportionally: a label delay of Δ samples leaves the buffers lagging the test distribution by Δ, and a labeled fraction $p < 1$ slows effective buffer growth by $1/p$ while leaving the frozen priors unaffected; the conformal calibration lags by the same Δ but retains its long-run coverage-tracking property. A systematic evaluation under delayed and partial labeling is left to future work.

Practical guidance: which components matter when. The ablations above admit a compact operational summary. The dual-buffer kNN correction is the dominant lever wherever temporal shift is pronounced (sberbank-housing, weather: 8.9–9.1% RMSE reduction from the Ens. to +kNN columns of Table 7); ensemble blending matters most when the prior pool is heterogeneous or the best single prior is uncertain a priori (classification, where it contributes up to +4.7 pp AUC (+8.2% relative) on

ecom-offers), whereas on maps-routing, where PCA reduction leaves the foundation-model priors weak, blending over them does not help (Table 7) and the effective remedy is to change the prior family rather than to blend harder; drift-aware blending adds value when drift is episodic rather than gradual (AdapBuf gains concentrate on sberbank-housing and weather in Table 13); the cascaded L2 layer acts as a safety backstop whose isolated contribution is small (≤ 0.03 pp AUC on classification, Table 10) but which guarantees the second pass cannot degrade accuracy; and ACI is required only when calibrated intervals are a deliverable. For deployments under tight computational budgets, a simplified configuration consisting of the frozen prior pool plus the dual-buffer kNN correction, with the cascaded L2 layer and the adaptive mechanisms disabled, already captures most of the full system's gain on 4 of 5 regression datasets (+kNN vs. Full in Table 7), and the remaining mechanisms are safe to enable incrementally because each self-prunes when unhelpful.

5.3.5. Hyperparameter sensitivity analysis

To assess robustness to hyperparameter choice, we conduct a one-at-a-time sensitivity analysis on the six principal fast-tier hyperparameters: the neighborhood size k, the retrieval temperature T, the correction scale s_1, the base blend ratio ω_0, the drift sensitivity α, and the drift window W, together with the two parameters of the cascaded second-layer correction (Section 4.3.4): its scale s_2 and its clipping multiplier c_2. Each is perturbed over $\{0.5, 0.7, 0.85, 1.0, 1.25, 1.5, 2.0\}\times$ its tuned value with all others held fixed, and the prequential metric is recomputed on a held-out probe stream (the first 40% of the test period, as in the sparsity analysis of Section B.1). This is a *relative* robustness re-evaluation of the already-fixed operating point, which was selected on the training-tail HPO validation stream (Section 5.1) and never on the test stream, rather than a second round of tuning; the absolute metrics below are probe-stream values and thus differ from the full-test numbers reported elsewhere, while the relative stability of interest is invariant to this choice. The L2 sweep mirrors the two-stage tuning protocol, in which the L2 parameters are optimized with the L1 stage frozen: it attaches the cascaded layer at its deployed settings to the frozen L1 stream and measures degradation relative to the cascaded operating point. The online benefit gate, which can only switch L2 off, is disabled during the sweep, so the reported L2 curves upper-bound the deployed sensitivity. The sweep reuses the pure-NumPy prequential simulator and cached predictions and runs in minutes on CPU. We report the regression instantiation (RMSE, 5 datasets) and the LimiX-based classification instantiation (AUC) used in Table 13.

Regression. Fig. 13 plots probe-stream RMSE relative to the tuned setting and Table 14 reports the worst-case relative degradation over the full 0.5×–2× range. Performance is stable across all six fast-tier hyperparameters: the largest degradation from any single perturbation is 1.75% (s_1 on weather) and the average worst-case degradation is 0.26%. The drift-mechanism hyperparameters (ω_0, α, W) are essentially inert ($\leq 0.11\%$), consistent with their self-regulating design that damps adaptation when the drift signal ρ_t stays near unity (Section 5.3); the dominant axis is the correction scale s_1, as expected since it directly multiplies the online correction, yet even a 2× change keeps RMSE within a few percent of the tuned setting. The two L2 parameters are flatter still: perturbing s_2 or c_2 degrades probe-stream RMSE by at most 0.11% (mean 0.04%), with smooth, near-flat curves around the tuned point. This is the graceful degradation anticipated by the second-order bound of Proposition 2.

Classification. The same analysis on the LimiX-based classification instantiation (AUC; homesite-insurance, ecom-offers, and homecredit-default) shows the identical pattern: the worst-case AUC drop from any single fast-tier perturbation is 2.38 pp (mean 0.29 pp), again concentrated on s_1 with ω_0, α, W inert. The L2 parameters remain benign as well, costing at most 0.66 pp (mean 0.26 pp) under the same protocol; in deployment the online benefit gate bounds this further, since it disables the layer outright whenever it ceases to help. The per-dataset classification figures and tables, together with the tuned operating points of both instantiations, are provided in Appendix D.

Table 14
Worst-case relative RMSE degradation (%) over the 0.5×–2× perturbation range, per hyperparameter and regression dataset (probe stream). Fast-tier parameters (top block) are measured relative to the L1 operating point, the cascaded-L2 parameters s_2, c_2 (bottom block) relative to the L1 + L2 operating point. Lower is more robust; the tuned setting is 0%.

Hyperparam.	Weather	Cooking	Delivery	Sberbank	Maps	Mean
k	0.40	0.08	0.13	0.23	0.11	0.19
T	1.40	0.61	0.11	0.05	0.14	0.46
s_1	1.75	0.62	0.90	0.83	0.26	0.87
ω_0	0.11	0.03	0.04	0.01	0.01	0.04
α	0.01	0.00	0.01	0.01	0.00	0.01
W	0.01	0.00	0.00	0.00	0.00	0.00
s_2	0.09	0.07	0.11	0.07	0.05	0.08
c_2	0.02	0.00	0.00	0.04	0.00	0.01

Table 15
ACI uncertainty quantification at 90% target coverage. Width: mean prediction interval width (lower is better). Cov. Std: standard deviation of rolling coverage across 10 temporal segments (lower is more stable). Width reduction is kNN-corrected vs. single prior.

	Mean Width			Cov. Std ($\times 10^{-3}$)		
Dataset	Single	Ens.	kNN	Single	kNN	Δ Width
Cooking	1.630	1.611	**1.576**	1.5	**1.1**	−3.3%
Sberbank	0.696	0.594	**0.464**	9.4	**4.3**	−33.3%
Weather	5.075	4.939	**4.480**	2.2	**1.0**	−11.7%
Delivery	2.017	1.903	**1.785**	1.6	**1.0**	−11.5%
Maps	0.558	0.549	**0.536**	0.8	**0.5**	−3.9%

5.4. Uncertainty quantification via adaptive conformal inference

The ACI framework described in Section 4.4 is evaluated with learning rate $\gamma = 0.005$, sliding window size $W_{\text{conf}} = 1000$, and target coverage $1 - \alpha_{\text{target}} = 90\%$. Unless otherwise noted, all experiments use the same kNN-corrected predictions from the full three-tier architecture that underlie the global ACI formulation in the method section.

5.4.1. Global ACI: interval efficiency and temporal stability

Table 15 summarizes prediction interval width and coverage stability across three prediction methods, all using ACI with $\gamma = 0.005$ and 90% target coverage.

The kNN-corrected predictions yield 3–33% narrower intervals and 1.4–2.2× more stable coverage, with the largest gains on the highest-shift datasets (sberbank-housing and weather). This confirms the mechanism proposed in Section 4.4: better point predictions translate directly into tighter conformal intervals at matched target coverage.

To examine whether this gain persists throughout the test stream rather than arising from a small subset of time periods, Fig. 14 visualizes the temporal evolution of prediction intervals on sberbank-housing and weather, the two datasets with the most pronounced distribution shift.

The width reduction from kNN correction is persistent across the entire test sequence, and the adaptive α_t trajectory serves as an implicit shift detector. In early segments, the kNN-corrected α_t can lie above the single-prior trajectory; this is expected under ACI's feedback dynamics (Eq. (19)) and reflects transient rebalancing of conservativeness rather than calibration failure. The key criterion is rolling coverage: as long as both trajectories remain centered around the 90% target, temporary ordering differences in α_t are benign. Both methods maintain coverage near the target, but kNN-corrected predictions achieve this with less interval width variation, indicating more stable calibration over time.

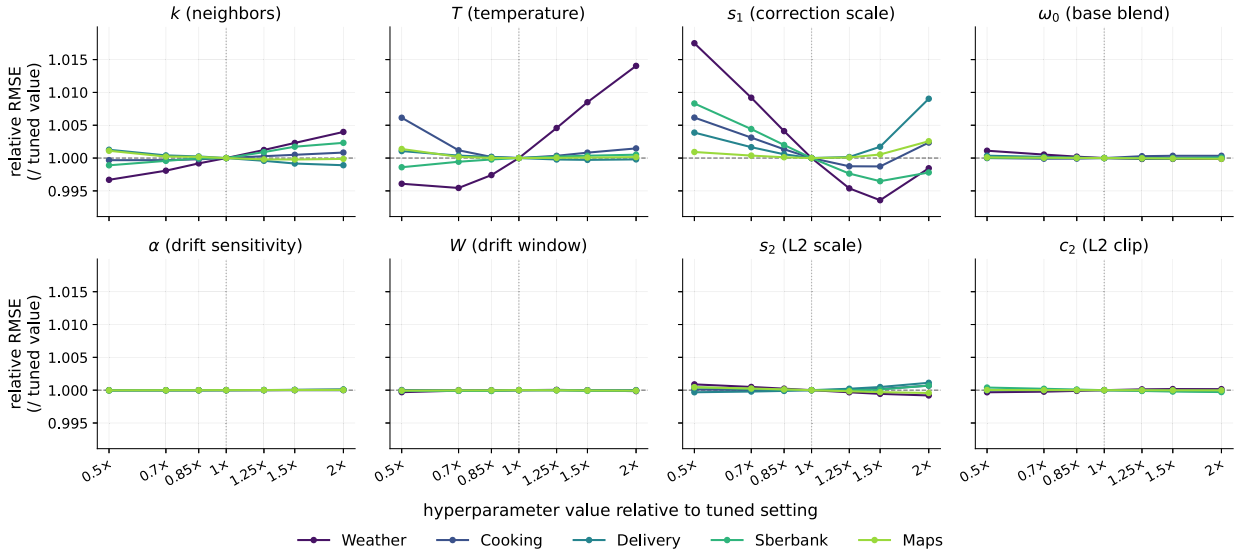

Fig. 13. Hyperparameter sensitivity (regression) on the held-out probe stream. Each panel perturbs one hyperparameter over 0.5×–2× its tuned value (dotted line at 1×); curves show probe-stream RMSE normalized by the tuned-setting RMSE (dashed line at 1.0). The six fast-tier panels are normalized by the L1 operating point and the two L2 panels (s_2, c_2) by the cascaded (L1 + L2) operating point. Flat curves near 1.0 indicate low sensitivity.

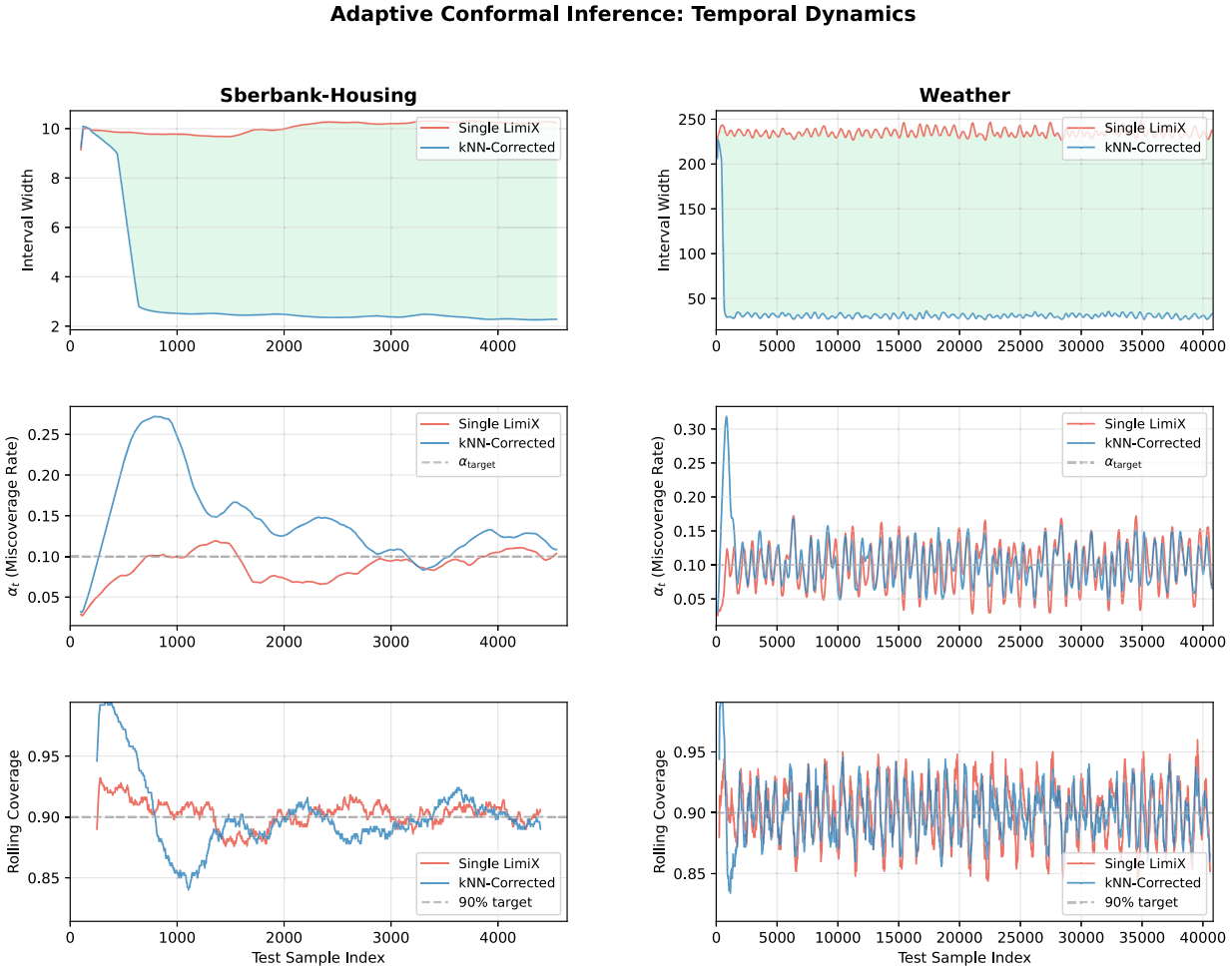

Fig. 14. Temporal dynamics of adaptive conformal inference. **Top:** Prediction interval width (kNN-corrected vs. single prior). **Middle:** Adaptive miscoverage rate α_t. **Bottom:** Rolling coverage (window = 500). All curves smoothed (200-sample average).

Table 16
Local vs. global conformal prediction at 90% target coverage. Width heterogeneity (std/mean) measures the degree to which interval width adapts to local prediction difficulty.

Dataset	Method	Coverage	Mean Width	Het.	Cov. Std
Cooking	Global ACI	90.0%	1.576	0.075	0.0011
	Local kNN + ACI	90.0%	1.558	**0.214**	**0.0007**
Sberbank	Global ACI	90.0%	0.464	0.216	0.0086
	Local kNN + ACI	89.9%	0.500	**0.558**	**0.0073**
Weather	Global ACI	90.0%	4.480	0.116	0.0013
	Local kNN + ACI	90.0%	4.398	**0.314**	**0.0009**
Delivery	Global ACI	90.0%	1.785	0.071	**0.0011**
	Local kNN + ACI	90.0%	1.800	**0.168**	**0.0011**
Maps	Global ACI	90.0%	0.536	0.090	**0.0005**
	Local kNN + ACI	90.0%	0.543	**0.194**	0.0012

5.4.2. Local kNN conformal extension for heteroscedastic uncertainty

While the global ACI results above demonstrate strong marginal calibration, they still rely on a single residual distribution per time step. We therefore evaluate the local kNN conformal extension from Section 4.4.2, using $k_{\text{conf}} = 50$ neighbors and a conformal buffer of $B_{\text{conf}} = 5000$. Three variants are compared: (i) Global ACI with a 1000-sample sliding window, (ii) Local kNN with fixed $\alpha = 0.1$, and (iii) Local kNN + ACI combining local quantiles with adaptive α adjustment. Table 16 reports the resulting trade-off between coverage, width, and heteroscedastic adaptation.

The local method produces 2–3× higher width heterogeneity, adapting interval width to local prediction difficulty rather than only to the global residual level. Local kNN + ACI also improves conditional coverage, i.e., coverage within prediction quintiles, reducing the maximum per-quintile deviation from 2.8–6.1 pp to 0.9–3.7 pp (Appendix C). In this sense, the local extension complements global ACI: the latter stabilizes marginal coverage under shift, while the former allocates uncertainty more selectively across the input space.

5.5. Generalization analysis

To evaluate generalization beyond TabReD, the three-tier architecture is tested on thirteen additional datasets with temporal or domain distribution shift (Table 17). Crucially, a hybrid prior configuration is introduced (XGB + LimiX): XGBoost trained on the full training set provides strong base predictions, while LimiX foundation model embeddings provide the representation manifold for fast-tier correction. This hybrid decouples prediction quality from manifold quality, addressing a key limitation of the original architecture where correction operated in the same representation space as the prior.

The architecture achieves competitive or state-of-the-art results on the additional benchmarks. Strong improvements are observed on high-dimensional datasets such as College Scorecard (+1.8 pp AUC, 269 features) and on Bike Sharing (−30.8% RMSE vs. LimiX, 12 features). The method also generalizes to domain shifts (Sepsis: +1.3 pp over TabPFNv2; Diabetes Readmission: +0.0107 over CatBoost) and low-dimensional settings (ACS datasets: +0.06–0.36 pp). On CIC-IDS-2017, TabPFNv2 achieves the highest AUC (0.9536), with the proposed method ranking second (0.9321); similarly, on BRFSS Diabetes, TabPFNv2 leads (0.8358) with the proposed method close behind (0.8333). MIMIC-IV ICU mortality prediction (Johnson et al., 2023) serves as a mild-shift control case: with 235 clinical features and a modest outcome shift (in-hospital mortality rising from 10.8% in the training period to 11.8% across the test period, and to 13.3% in its final segment, the 2020–2022 COVID-19 window), the architecture achieves 0.9320 AUC against hyperparameter-tuned tree baselines (XGBoost 0.9302, CatBoost 0.9300, LightGBM 0.9301, the last not tabulated), a margin of 0.18 pp, demonstrating graceful behavior when shift is minimal. Combined with the TabReD results, the full three-tier architecture achieves the best reported result on 17 of 21 benchmarks spanning temporal, domain, and demographic distribution shifts, on three of them (ecom-offers, maps-routing, and homecredit-default relative to its strongest static baseline) by margins that Section 5.2 reports as statistical ties.

5.6. Computational efficiency

A key advantage of the proposed architecture is its computational efficiency at test time. Unlike TARS (Wang et al., 2026) and temporal encoding methods (Cai & Ye, 2025b), which require end-to-end retraining of the backbone network for each dataset, and online fine-tuning methods, which require GPU-based gradient updates during deployment, the proposed method performs online adaptation through the full three-tier inference pipeline on CPU, with the fast tier implemented as non-parametric correction over frozen representations.

Table 18 reports wall-clock times for the online adaptation phase across seven of the eight TabReD datasets (maps-routing excluded due to differing cache structure). All experiments were conducted on a single machine with an Intel Core i9-14900K CPU and NVIDIA GeForce RTX 5090 GPU. For TARS, the reported training time corresponds to a 3-layer MLP backbone with three temporal encoding pathways (100 epochs), which must be repeated for each backbone architecture. Online FT-Transformer trains for 20 epochs on the training set, then fine-tunes on each revealed test batch.

Table 17
Additional temporal and domain-shift benchmarks. Classification: AUC↑; Regression: RMSE↓. Best in **bold**, second-best underlined.

Dataset	Feat.	Shift Type	XGBoost	CatBoost	LimiX	TabPFNv2	Ours
Classification (AUC↑)							
ACS Income	10	Temporal (2014–17→2018)	0.8888	0.8876	0.8660	0.8831	**0.8924**
ACS Employment	16	Temporal (2014–17→2018)	0.9040	0.9040	0.8870	0.9026	**0.9046**
ACS PublicCoverage	19	Temporal (2014–17→2018)	0.8241	0.8242	0.7630	0.8137	**0.8271**
ACS Unemployment	16	Temporal (2014–17→2018)	0.8253	0.8243	0.7500	0.8196	**0.8268**
Credit Card Fraud	29	Temporal (time-sorted)	0.9069	0.9377	0.9560	0.9782	**0.9815**
Sepsis (PhysioNet)	174	Domain (Hospital A→B)	0.8994	0.8992	0.8810	0.9061	**0.9188**
CIC-IDS-2017	68	Temporal (Mon–Wed→Thu–Fri)	0.7724	0.7078	0.8599	**0.9536**	0.9321
College Scorecard	269	Temporal (≤2010→2011–13)	0.9287	0.9253	0.7795	0.8897	**0.9469**
IEEE-CIS Fraud	434	Temporal (time-sorted)	0.8859	0.8767	0.8771	0.8281	**0.8894**
Diabetes Readmission	43	Domain (Emergency→Elective)	0.6632	0.6777	0.6105	0.6457	**0.6884**
BRFSS Diabetes	28	Domain (White→non-White)	0.8217	0.8308	0.8152	**0.8358**	0.8333
MIMIC-IV Mortality	235	Temporal (2008–16→17–22)	0.9302	0.9300	0.8732	0.9060	**0.9320**
Regression (RMSE↓)							
Bike Sharing	12	Temporal (time-sorted)	136.74	129.61	111.08	124.03	76.91

Table 18
Computational efficiency. **Ours**, **Streaming FM**, **Online FT-T**, and **Online XGB** report the *online/deployment* phase: wall-clock seconds for the full test stream and the corresponding per-sample cost (ms/sample = seconds × 1000 / test size, Table 2). **TARS** time is one-time backbone *training* over the training set, reported as total seconds, so a per-test-sample value is not applicable (†). The proposed online phase runs on CPU with no gradient updates to the frozen priors, and its one-time setup (prior-cache generation 5–30 min; Optuna search ∼83 min, CPU) is amortized over deployment.

	Ours		Streaming FM		Online FT-T		Online XGB		TARS	
Dataset	(s)	ms/sm	(s)	ms/sm	(s)	ms/sm	(s)	ms/sm	(s)	ms/sm†
cooking-time	**1.7**	0.041	26.29	0.631	30.6	0.735	13.7	0.329	125.8	—
sberbank-housing	**0.2**	0.043	4.46	0.960	3.2	0.689	1.1	0.237	10.9	—
delivery-eta	**1.4**	0.038	24.50	0.663	43.4	1.175	12.5	0.339	157.2	—
weather	**1.6**	0.039	48.89	1.197	46.0	1.126	10.6	0.260	192.2	—
homesite-ins.	**0.8**	0.049	14.34	0.880	37.1	2.277	5.6	0.344	132.4	—
ecom-offers	**1.1**	0.042	13.27	0.502	17.8	0.673	4.7	0.178	63.8	—
homecredit-def.	**2.7**	0.048	84.30	1.505	44.1	0.787	32.5	0.580	165.5	—
Mean	**1.4**	**0.043**	30.9	0.906	31.7	1.066	11.5	0.324	121.1	—
GPU req.	No		Yes		Yes		No		Yes	
Speedup	1×		22× slower		23× slower		8× slower		86× (training)	

The proposed method completes online adaptation in 0.2–2.7 seconds per dataset (mean 1.4 s), compared to 3.2–46.0 s for Online FT-Transformer (mean 31.7 s, 23× slower), 1.1–32.5 s for Online XGB Retrain (mean 11.5 s, 8× slower), and 10.9–192.2 s for TARS training (mean 121.1 s, 86× slower). Crucially, the proposed method requires no GPU during the online phase, operating entirely on CPU through the frozen-prior, lightweight blending, and non-parametric fast-tier update pipeline. TARS and Online FT-Transformer both require GPU access for gradient computation.

The one-time setup costs for the proposed method include: (1) prior model training or cache generation (5–30 min per dataset, GPU, shared with any method that uses pretrained models), and (2) Optuna hyperparameter search over kNN parameters (∼500 trials at ∼10 s/trial ≈ 83 min, CPU only, no gradient computation). These setup costs are amortized over the deployment lifetime and are comparable to or lower than TARS's requirement to retrain each backbone–temporal-encoding combination from scratch. In effect, the method shifts cost from deployment-time gradient updates to offline validation-time hyperparameter search; the online phase is cheap because that search is paid once, before deployment. Test-time local training (TTLT) (Kang & Kang, 2026) is more expensive still: because it fine-tunes a fresh local model for each query, it incurs 40–47 ms per test sample on GPU (mean ∼44 ms), roughly three orders of magnitude above the proposed method's 0.04 ms gradient-free correction, and likewise depends on a GPU at deployment time. Streaming TabPFN, the closest competitor in accuracy on the low-shift datasets, is about 21× slower per test sample on average (12–31× per dataset, Table 18; the 22× in that table's speedup row is the corresponding ratio of total wall-clock times): it performs a full foundation-model forward pass over its sliding context window for every prediction, at ∼0.91 ms per test sample on a GPU, versus the proposed method's 0.043 ms CPU-only update, and the ordering holds even on the datasets where its accuracy is closest (cooking-time 15×, sberbank-housing 22×). Its narrow accuracy gap on foundation-model-friendly datasets thus comes with both a GPU requirement and a substantially higher per-sample cost.

6. Conclusions

This paper presented a three-tier architecture for tabular prediction under temporal distribution shift, motivated by the hierarchical timescales of prefrontal cortex plasticity. The architecture combines frozen pretrained priors, lightweight multi-prior reconfiguration, and online non-parametric correction, while supporting task-specific instantiations for regression and classification under the same high-level decomposition.

Across twenty-one datasets, including eight TabReD benchmarks and thirteen additional temporal or domain-shift tasks, the architecture delivers consistent gains over strong baselines. On TabReD, the best configuration outperforms all published baselines on four of five regression datasets (on three of them by margins beyond run-to-run variation; the maps-routing margin falls within it), and achieves classification AUC of 0.9652, 0.8697, and 0.6373: where shift is pronounced (homecredit-default) it exceeds the best end-to-end-retrained temporal baseline by +2.4 pp, a margin of roughly five DeLong standard errors, while on the two near-stationary datasets it is statistically tied with the strongest retrained baselines (differences of +0.0001 and −0.0010 AUC, i.e. 0.03 and 0.69 standard errors of a single AUC estimate; Section 5.2). Fast-tier kNN correction is the most impactful regression component (2.3–9.1% RMSE reduction), and the online adaptation runs entirely on CPU with no gradient updates to the frozen priors (the optional low-rank embedding adapter is the only gradient-based component and is frequently disabled), yielding a 23× speedup over online fine-tuning and 86× relative to TARS's one-time training under the reported setup (Section 5.6). On top of these point predictions, adaptive conformal inference yields 3–33% narrower intervals at equivalent coverage on the regression benchmarks, with 1.4–2.2× more stable coverage under shift.

Beyond TabReD, the architecture ranks first on 11 of 13 additional benchmarks and second on the remaining two. A hybrid configuration that combines XGBoost predictions with LimiX foundation-model embeddings for kNN correction produces particularly strong improvements under genuine shift, showing that prediction quality and embedding quality can be decoupled effectively.

Several limitations remain. The current approach relies on precached foundation-model predictions, requires per-dataset Optuna tuning, and is less advantageous on low-dimensional or near-stationary datasets where post-hoc correction offers limited lift. Future work will explore meta-learned hyperparameter initialization, adaptive prior selection, and broader temporal benchmarks. The MIMIC-IV result suggests that the architecture also behaves gracefully in mild-shift settings, supporting its robustness across a range of distribution-shift regimes.

CRediT authorship contribution statement

Tianyu Wang: Writing – review & editing, Writing – original draft, Project administration, Methodology, Conceptualization; **Mian Li:** Writing – review & editing, Supervision, Funding acquisition; **Xi Vincent Wang:** Writing – review & editing, Supervision; **Lihui Wang:** Writing – review & editing, Supervision.

Declaration of generative AI

During the preparation of this work, the authors used generative AI tools (OpenAI ChatGPT and Anthropic Claude) to assist in the preparation of the manuscript and to improve language and readability. After

using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The work presented here was supported by Swedish research center of eXcellence in PRoduction RESearch (XPRES) and National Natural Science Foundation of China (Grant No. 52275263).

Data availability

The complete source code is publicly available at https://github.com/gunnerwang/Beyond-Temporal-Encoding, including the implementation of all three tiers, the scripts that rebuild the cached prior predictions and embeddings used in the experiments, the per-dataset configuration files with the tuned hyperparameter values of every reported operating point, the Optuna search scripts, and the evaluation scripts that reproduce all tables and figures in this manuscript. The datasets analyzed in this work are derived from publicly available sources, including the TabReD benchmark and additional public temporal and domain-shift benchmarks described in the manuscript.

Appendix A. Full-model verification

Table A.1
Simulator–full-model RMSE alignment (7-way + kNN).

Dataset	Simulator	Full model	Gap
Cooking-time	0.47578	0.47579	+0.00001
Sberbank-housing	0.22176	0.22209	+0.00033
Delivery-eta	0.54427	0.54426	−0.00001
Weather	1.44228	1.44544	+0.00316
Homesite-ins.	0.29604	0.29607	+0.00003
Ecom-offers	0.46506	0.46513	+0.00007

Table A.1 verifies that the hyperparameters tuned on the lightweight prequential simulator transfer faithfully to the full model.

All gaps are below 0.004, confirming faithful reproduction of the full inference pipeline; the slightly larger weather gap reflects its larger context size ($C = 10{,}000$).

Appendix B. Ensemble weight and sparsity analysis

Minimum-variance weights vs. Optuna weights. Table B.1 compares the Optuna-tuned ensemble weights with the closed-form minimum-variance solution (Eq. 5) and simpler baselines.

Table B.1
Ensemble RMSE under different weighting schemes; $\cos(\boldsymbol{w}_{\text{opt}}, \boldsymbol{w}_{\text{mv}})$ denotes Optuna–minimum-variance weight similarity.

Dataset	Equal	MinVar	Optuna	Best Single	cos
Cooking	0.4904	0.4882	**0.4878**	0.4891	0.95
Sberbank	0.2428	**0.2378**	0.2393	0.2400	0.95
Weather	1.6102	**1.5665**	1.5786	1.5824	0.93
Delivery	0.5642	**0.5608**	0.5627	0.5633	0.80
Maps	0.1759	0.1777	**0.1726**	0.1723	0.14

Optuna and minimum-variance weights achieve similar RMSE on four of five datasets (cosine similarity ≥ 0.80), indicating a well-behaved weight landscape; inter-scheme RMSE differences remain below 3%.

B.1. Ensemble sparsity analysis

Because LimiX priors are highly correlated ($\rho > 0.95$), Table B.2 tests whether a sparse subset suffices using greedy forward selection on the first 40% of the test period and evaluation on the remaining 60%.

Table B.2
Ensemble sparsity analysis. Val-FS: validation-based forward selection; parentheses give subset size.

	Fixed Weights Only			With Online Adaptation		
Dataset	Full	Val-FS	Δ	Full	Val-FS	Δ
Cooking	0.4878	**0.4841** (2)	−0.8%	**0.4738**	0.4826 (2)	+1.9%
Sberbank	0.2393	0.2403 (4)	+0.4%	**0.2185**	0.2242 (4)	+2.6%
Delivery	0.5627	**0.5608** (5)	−0.3%	**0.5443**	0.5487 (5)	+0.8%
Weather	1.5786	**1.5515** (2)	−1.7%	**1.4237**	1.4812 (2)	+4.0%
Maps	0.1726	**0.1696** (2)	−1.7%	**0.1680**	0.1727 (2)	+2.8%

Sparse subsets outperform the full ensemble on 4/5 datasets under fixed weights alone, but the full 7-prior ensemble of this analysis (six LIMIX variants plus XGBoost) wins by 0.8–4.0% with online adaptation because low-weight priors still enrich the kNN correction space. Thus, pruning must account for the downstream fast tier, not only the blending layer. The sparse subsets reuse hyperparameters tuned for the full ensemble, so this diagnostic analysis is a conservative probe of prunability rather than a configuration-selection step.

Appendix C. Conditional coverage analysis

Table C.1 compares conditional coverage across prediction quintiles.

Table C.1
Conditional coverage by prediction quintile (target: 90%); the last column reports maximum deviation.

Dataset	Method	Q1	Q2	Q3	Q4	Q5	Max Dev.
Cooking	Global ACI	87.2	86.8	88.3	92.2	95.5	5.5pp
	Local kNN + ACI	90.0	89.5	89.4	90.1	90.9	**0.9pp**
Sberbank	Global ACI	92.7	94.1	86.8	90.3	86.0	4.1pp
	Local kNN + ACI	88.4	89.2	88.2	91.6	92.0	**2.0pp**
Weather	Global ACI	91.1	92.8	91.6	87.3	87.3	2.8pp
	Local kNN + ACI	90.1	91.0	90.2	89.3	89.4	**1.0pp**
Delivery	Global ACI	88.1	87.9	89.7	91.4	92.9	2.9pp
	Local kNN + ACI	88.7	88.7	89.5	90.9	92.2	**2.2pp**
Maps	Global ACI	94.1	92.7	90.6	88.7	83.9	6.1pp
	Local kNN + ACI	92.6	91.5	90.3	89.3	86.3	**3.7pp**

Local kNN + ACI reduces the maximum per-quintile deviation from 2.8–6.1 pp to 0.9–3.7 pp, showing that the embedding space captures heteroscedastic uncertainty.

Appendix D. Sensitivity analysis details

This appendix complements Section 5.3.5. Table D.1 reports regression operating points, while Fig. D.1 and Tables D.2–D.3 give the classification sensitivity results and operating points. Homecredit-default's LimiX cache was regenerated at ctx=3000 with the full seven-prior ensemble, yielding a probe-stream AUC (0.819) close to the tuned operating point (0.823).

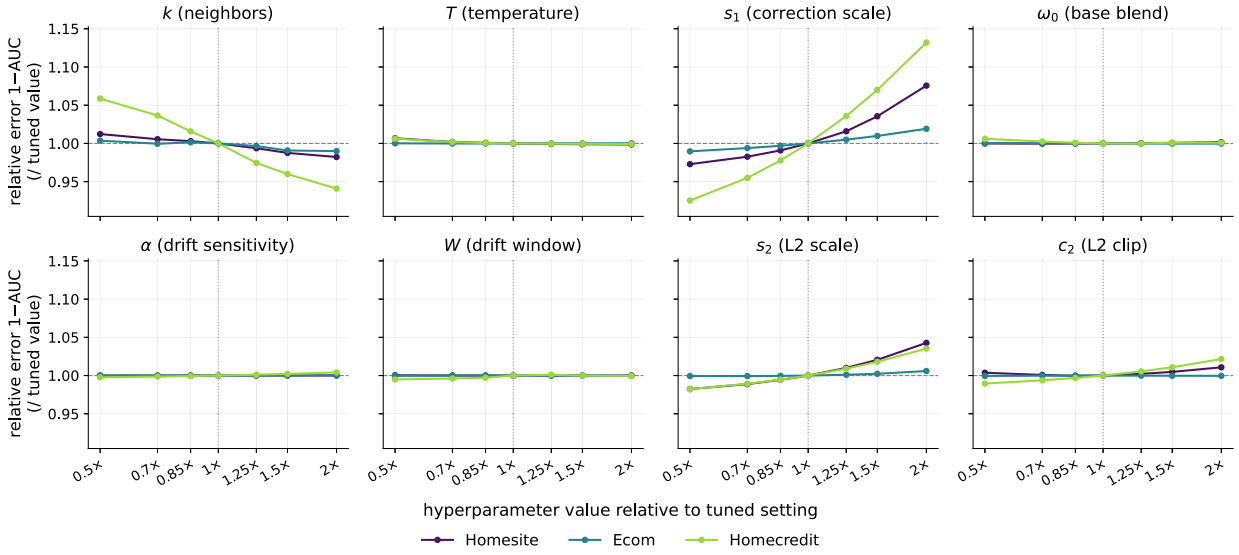

Fig. D.1. Classification hyperparameter sensitivity on the held-out probe stream. Curves show relative error $1 - \mathrm{AUC}$ normalized by the tuned setting; the s_2 and c_2 panels use the cascaded L1 + L2 operating point.

Table D.1
Regression sensitivity operating points: tuned hyperparameters and probe-stream RMSE. T is T_S/T_L; s_2, c_2 are cascaded-L2 settings.

Dataset	k	T_S/T_L	s_1	ω_0	α	W	s_2	c_2	Probe RMSE
Weather	48	1.0/1.4	1.17	0.74	1.95	192	0.51	3.47	1.4955
Cooking	167	18.7/1.2	1.12	0.76	0.51	512	0.20	1.04	0.4701
Delivery	71	6.5/2.0	1.12	0.49	2.98	320	0.15	1.35	0.5603
Sberbank	24	8.2/10.2	1.18	0.38	2.14	320	0.55	0.57	0.2176
Maps	66	9.2/1.4	1.14	0.20	5.91	64	0.26	0.45	0.1680

Table D.2
Worst-case AUC drop (pp) over 0.5×–2× hyperparameter perturbations for classification; lower is better and the tuned setting is 0 pp.

Hyperparam.	Homesite	Ecom	Homecredit	Mean
k	0.058	0.144	1.059	0.420
T	0.032	0.007	0.113	0.051
s_1	0.358	0.755	2.382	1.165
ω_0	0.008	0.017	0.108	0.044
α	0.001	0.006	0.073	0.027
W	0.001	0.000	0.018	0.006
s_2	0.209	0.233	0.660	0.367
c_2	0.053	0.000	0.403	0.152

Table D.3
Classification sensitivity operating points: tuned hyperparameters and probe-stream AUC. s_2, c_2 are cascaded-L2 settings.

Dataset	k	T_S/T_L	s_1	ω_0	α	W	s_2	c_2	Probe AUC
Homesite	180	2.2/5.4	0.30	0.41	1.00	384	0.16	0.71	0.9526
Ecom	177	16.1/12.0	0.48	0.29	7.97	384	0.06	1.94	0.6061
Homecredit	142	6.2/7.3	0.32	0.70	2.42	448	0.07	2.09	0.8195

Appendix E. Paired verification of the adaptive mechanisms

This appendix verifies that the low-rank embedding adapter of Section 4.3.5 is an active, trained component. All measurements are paired and deterministic: every hyperparameter other than the adapter is frozen at the configuration recorded for each dataset, so adapter-on and adapter-off runs differ in nothing else and no re-tuning of the surrounding pipeline is involved. We run the full test streams of weather, cooking-time, and delivery-eta (40,840/41,648/36,927 samples; 638/651/577 updates at a chunk size of 64) over 192-dimensional PCA-whitened LimiX embeddings, with the same LimiX-plus-XGBoost prior ensemble and hyperparameters held fixed across the adapter-on and adapter-off arms of every cell (seven priors on all three datasets; on cooking-time this is seven of the eight priors used in the main text, one LimiX seed variant being absent from the cache), initializing $\boldsymbol{U} \sim \mathcal{N}(0, \sigma_0^2)$ with $\sigma_0 = 1/\sqrt{3d} \approx 0.0417$ and $\boldsymbol{V} = \boldsymbol{0}$, sweeping rank $r \in \{4, 16\}$ and η over 10^{-5}–10^{1} with five initialization seeds per cell.

Table E.1
Adapter activation on the weather stream: stream-end Frobenius norms of the learned low-rank transform and of the drift of the first factor from its random initialization. The step sizes listed lie inside the deployed search range 10^{-5}–10^{-2} (Table 4). These norms and those in Table E.2 come from separate sweeps and are not directly comparable cell to cell.

r	η	$\|\boldsymbol{U}\boldsymbol{V}^\top\|_F$ (stream end)	$\|\boldsymbol{U} - \boldsymbol{U}_0\|_F$ (stream end)
4	10^{-5}	1.70×10^{-3}	6.59×10^{-5}
4	10^{-4}	1.70×10^{-2}	4.79×10^{-3}
4	10^{-3}	8.69×10^{-1}	7.66×10^{-1}
16	10^{-5}	3.74×10^{-3}	1.38×10^{-4}
16	10^{-4}	3.75×10^{-2}	1.07×10^{-2}
16	10^{-3}	1.14×10^{1}	2.42

Activation is straightforward to verify. Table E.1 reports, on the weather stream, the stream-end Frobenius norm of the learned transform, $\|\boldsymbol{U}\boldsymbol{V}^\top\|_F$, and of the drift of the first factor from its random initialization, $\|\boldsymbol{U} - \boldsymbol{U}_0\|_F$, over the low-$\eta$ part of the grid, which lies inside the deployed search range 10^{-5}–10^{-2} of Table 4: $\|\boldsymbol{U}\boldsymbol{V}^\top\|_F$ departs from zero in every configuration, and both factors move ($\boldsymbol{V}$ leaves zero at the first update, since $\nabla_{\boldsymbol{V}} \mathcal{L} \propto \boldsymbol{U}_0 \neq \boldsymbol{0}$, after which $\boldsymbol{U}$ follows), so the learned transform is a trained low-rank adaptation rather than a fixed random projection, while $\boldsymbol{U}\boldsymbol{V}^\top = \boldsymbol{0}$ at $t = 0$ preserves the initial embedding geometry exactly. Table E.2 reports the paired effect at the best probed cell per dataset, against both the no-adapter baseline and a matched-perturbation control in which $\boldsymbol{U}, \boldsymbol{V}$ are frozen at random values rescaled to the same $\|\boldsymbol{U}\boldsymbol{V}^\top\|_F$ reached by the learned run. The adapter significantly outperforms both on all three datasets (the weather and cooking-time comparisons survive a conservative Bonferroni correction across the 14-cell rank × learning-rate grid; the smaller delivery-eta effect is nominally significant, and its case rests on the dose–response pattern below rather than the single cell); beating the matched control shows the improvement comes from the learned *direction*, not from perturbing

Table E.2
Paired adapter effect per dataset (five initialization seeds per cell; all other hyperparameters frozen). The selection rule is stated explicitly: among all swept cells passing both significance thresholds below, the cell with the largest improvement is reported (4/2/3 cells pass per dataset). Cells with larger raw improvement but seed variability exceeding the effect are excluded by the filter (e.g. weather $r = 4$, $\eta = 1$: $\Delta = -7.4 \times 10^{-4}$, seed SD 1.1×10^{-3}, $t = -1.5$). t vs. baseline: one-sample t over the seeded runs against the deterministic no-adapter value (df = 4; $|t| > 2.78$ for $p < 0.05$). t vs. random: two-sample t against the matched-perturbation control (df = 8; $|t| > 2.31$). Every cell clears both thresholds.

Dataset	r	η	$\|UV^\top\|_F$	Baseline RMSE	Adapter RMSE	t vs. baseline	t vs. random
weather	16	10^{-1}	1.93×10^{-1}	1.435197	1.434858	−6.5	−5.4
cooking-time	16	10^{1}	7.67×10^{-1}	0.474267	0.474197	−8.8	−5.6
delivery-eta	16	10^{-1}	1.22×10^{-2}	0.546660	0.546652	−4.8	−4.2

the retrieval geometry per se. The gain follows a dose–response curve, growing monotonically with η over three decades (with divergence at the largest step size on weather), and no run diverges at $\eta \leq 10^{-1}$ on any dataset, rank, or key-handling mode; a key-handling control (storing projected vs. raw whitened embeddings as buffer keys) is immaterial at usable step sizes (median difference $\leq 2.2 \times 10^{-5}$). The absolute effect is small (relative RMSE reductions of 0.001–0.06% across the probed non-divergent cells), consistent with the adapter's role as an optional, supporting mechanism; the deployed η search range remains the conservative, stability-first choice described in Section 4.3.5.

The same paired protocol applies to the drift-aware blending mechanism. With the deployed configuration frozen and only the blend rule switched between the adaptive ω_t of Eq. (12) and the static $\omega_0/2$ that it reduces to at $\alpha = 0$, the comparison is deterministic and carries neither seed nor search variance. On sberbank-housing, the dataset with the strongest temporal shift, the mechanism yields a paired improvement of 1.69×10^{-4} RMSE (0.077%), of the same order as the adapter's isolated effect. The blend ratio itself varies materially along the stream (e.g. from 0.069 to 0.614 on cooking-time), the behavior illustrated in Fig. 12. A varying blend ratio does not by itself imply an accuracy gain: on the near-stationary datasets the drift signal stays close to unity and the mechanism correctly defaults to near-static behavior, as discussed in Section 5.3.

The adapter study has likewise been extended to sberbank-housing under a protocol identical to that used for the other datasets: the same initialization $\boldsymbol{U} \sim \mathcal{N}(0, \sigma_0^2)$ with $\sigma_0 = 1/\sqrt{3d}$ and $\boldsymbol{V} = \boldsymbol{0}$, the same rank and learning-rate grid, five initialization seeds per cell, and the same matched-perturbation and key-handling controls; the findings are consistent with those reported above.

References

Aguiar, G., Krawczyk, B., & Cano, A. (2024). A survey on learning from imbalanced data streams: Taxonomy, challenges, empirical study, and reproducible experimental framework. *Machine Learning, 113* (7), 4165–4243.

Aguiar, G. J., & Cano, A. (2024). A comprehensive analysis of concept drift locality in data streams. *Knowledge-Based Systems, 289*, 111535. https://doi.org/10.1016/j.knosys.2024.111535

Bayram, F., Ahmed, B. S., & Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems, 245*, 108632. https://doi.org/10.1016/j.knosys.2022.108632

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint arXiv:1308.3432.

Bifet, A., & Gavaldà, R. (2009). Adaptive learning from evolving data streams. In *International symposium on intelligent data analysis* (pp. 249–260). Springer.

Cai, H.-R., & Ye, H.-J. (2025a). Feature-aware modulation for learning from temporal tabular data. arXiv preprint arXiv:2512.03678.

Cai, H.-R., & Ye, H.-J. (2025b). Understanding the limits of deep tabular methods with temporal shift. In *Proceedings of the 42nd International conference on machine learning (ICML)* (pp. 6366–6386). (*vol. 267*). PMLR.

Cai, Z., Bai, G., Jiang, R., Song, X., & Zhao, L. (2024). Continuous temporal domain generalization. In *Advances in neural information processing systems* (pp. 127987–128014). (*vol. 37*).

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International conference on knowledge discovery and data mining* (pp. 785–794).

Dawid, A. P. (1984). Present position and potential developments: Some personal views: Statistical theory: The prequential approach. *Journal of the Royal Statistical Society: Series A (General), 147* (2), 278–292.

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44* (3), 837–845.

Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. In *Proceedings of the sixth ACM SIGKDD International conference on knowledge discovery and data mining* (pp. 71–80).

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29* (5), 1189–1232.

Gardner, J., Popovic, Z., & Schmidt, L. (2023). Benchmarking distribution shift in tabular data with tableshift. *Advances in Neural Information Processing Systems, 36*, 53385–53432.

Gibbs, I., & Candès, E. (2021). Adaptive conformal inference under distribution shift. In *Advances in neural information processing systems* (pp. 1660–1672). (*vol. 34*).

Gomes, H. M., Bifet, A., Read, J., Barddal, J. P., Enembreck, F., Pfharinger, B., Holmes, G., & Abdessalem, T. (2017). Adaptive random forests for evolving data stream classification. *Machine Learning, 106* (9), 1469–1495.

Gorishniy, Y., Kotelnikov, A., & Babenko, A. (2024a). TabM: Advancing tabular deep learning with parameter-efficient ensembling. arXiv preprint arXiv:2410.24210.

Gorishniy, Y., Rubachev, I., & Babenko, A. (2022). On embeddings for numerical features in tabular deep learning. In *Advances in neural information processing systems*.

Gorishniy, Y., Rubachev, I., Kartashev, N., Shlenskii, D., Kotelnikov, A., & Babenko, A. (2024b). TabR: Tabular deep learning meets nearest neighbors. In *International conference on learning representations (ICLR)*. https://openreview.net/forum?id=rhgIgTSSxW.

Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. arXiv preprint arXiv:2106.11959.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35*, 507–520.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction. (2nd ed.). Springer.

Helli, K., Schnurr, D., Hollmann, N., Müller, S., & Hutter, F. (2024). Drift-resilient tabPFN: In-context learning temporal distribution shifts on tabular data. *Advances in Neural Information Processing Systems, 37*, 98742–98781.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeister, R. T., & Hutter, F. (2025). Accurate predictions on small data with a tabular foundation model. *Nature, 637* (8045), 319–326.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International conference on learning representations (ICLR)*.

Jin, Y., Yang, Z., Chu, X., & Ma, L. (2024). Temporal domain generalization via learning instance-level evolving patterns. In *Proceedings of the thirty-third international joint conference on artificial intelligence* (pp. 4255–4263). https://www.ijcai.org/proceedings/2024/0470.pdf.

Johnson, A. E. W., Bulgarelli, L., Shen, L., Gayles, A., Shammout, A., Horng, S., Pollard, T. J., Hao, S., Moody, B., Gow, B., Lehman, L. H., Celi, L. A., Mark, R. G. (2023). MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data, 10* (1), 1.

Kang, M., & Kang, S. (2026). Test-time local training of neural network for tabular data. *Scientific Reports, 16*, 1863.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in neural information processing systems*. (*vol. 30*).

Kim, C., Kim, T., Woo, S., Yang, J. Y., & Yang, E. (2024). Adaptable: Test-time adaptation for tabular data via shift-aware uncertainty calibrator and label distribution handler. arXiv preprint arXiv:2407.10784.

Kirichenko, P., Izmailov, P., & Wilson, A. G. (2023). Last layer re-training is sufficient for robustness to spurious correlations. In *International conference on learning representations* (pp. 1–20). https://openreview.net/forum?id=Zb6c8A-Fghk.

Krogh, A., & Vedelsby, J. (1994). Neural network ensembles, cross validation, and active learning. In *Advances in neural information processing systems* (pp. 231–238). (*vol. 7*).

Li, R. et al. (2025). Faith: Frequency-domain attention in two horizons for time series forecasting. *Knowledge-Based Systems, 309*, 112790. https://doi.org/10.1016/j.knosys.2024.112790

Lourenço, A., Rodrigo, J., Gama, J., & Marreiros, G. (2025). In-context learning of evolving data streams with tabular foundational models. arXiv preprint arXiv:2502.16840.

Ma, J. (2024). Improved self-training for test-time adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 23701–23710).

Miller, J. A., & Constantinidis, C. (2024). Timescales of learning in prefrontal cortex. *Nature Reviews Neuroscience*, *25* (9), 597–610.

Pellicani, A., & Ceci, M. (2025). Positional trace encoding for next activity prediction in event logs. *Knowledge-Based Systems*, *319*, 113544. https://doi.org/10.1016/j.knosys.2025.113544

Perich, M. G., Narain, D., Gallego, J. A. (2025). A neural manifold view of the brain. *Nature Neuroscience*, *28*, 1582–1597.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). Catboost: Unbiased boosting with categorical features. In *Advances in neural information processing systems*. (*vol. 31*).

Qu, J., Holzmüller, D., Varoquaux, G., & Le Morvan, M. (2025). TabICL: A tabular foundation model for in-context learning on large data. In *International conference on machine learning (ICML)*.

Rubachev, I., Kartashev, N., Gorishniy, Y., & Babenko, A. (2025). TabReD: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks. In *International conference on learning representations (ICLR)*. Spotlight.

Sun, Y., Li, Y., Weng, P., & Chen, J. (2025). Test-time local training with foundation models. *Nature Communications*, *16*, 1208.

Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., & Ng, R. (2020). Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, *33*, 7537–7547.

Tartakovsky, A., Nikiforov, I., & Basseville, M. (2014). Sequential analysis: Hypothesis testing and changepoint detection. CRC Press.

Vela, D., Sharp, A., Zhang, R., Nguyen, T., Hoang, A., & Pianykh, O. S. (2022). Temporal quality degradation in AI models. *Scientific Reports*, *12* (1), 11654.

Villaizán-Vallelado, M., Salvatori, M., Carro, B., & Sanchez-Esguevillas, A. J. (2024). Graph neural network contextual embedding for deep learning on tabular data. *Neural Networks*, *173*, 106180. https://doi.org/10.1016/j.neunet.2024.106180

Wang, D., Shelhamer, E., Liu, S., Olshausen, B., & Darrell, T. (2020). Tent: Fully test-time adaptation by entropy minimization. arXiv preprint arXiv:2006.10726.

Wang, Q., Fink, O., Van Gool, L., & Dai, D. (2022). Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition* (pp. 7201–7211).

Wang, T., Zhang, M., Lu, M., & Li, M. (2026). Multi-timescale representation with adaptive routing for deep tabular learning under temporal shift. *Neural Networks*, *199*, 108670. https://doi.org/10.1016/j.neunet.2026.108670

Xu, F., Deng, S., Jia, T., Yu, X., & Chen, D. (2024). Ensembling disentangled domain-specific prompts for domain generalization. *Knowledge-Based Systems*, *301*, 112358. https://doi.org/10.1016/j.knosys.2024.112358

Yang, H., Wang, M., Yu, Z., Zhang, H., & Zhou, Y. (2024). Confidence-based and sample-reweighted test-time adaptation. *Knowledge-Based Systems*, *283*, 111164. https://doi.org/10.1016/j.knosys.2023.111164

Yao, H., Choi, C., Cao, B., Lee, Y., Koh, P. W. W., & Finn, C. (2022). Wild-time: A benchmark of in-the-wild distribution shift over time. *Advances in Neural Information Processing Systems*, *35*, 10309–10324.

Ye, H.-J., Liu, S.-Y., Cai, H.-R., Zhou, Q.-L., & Zhan, D.-C. (2025). A closer look at deep learning methods on tabular datasets. arXiv preprint arXiv:2407.00956.

Ye, H.-J., Zhou, Q., Yin, H.-H., Zhan, D.-C., & Chao, W.-L. (2023). Rethinking pre-training in tabular data: A neighborhood embedding perspective. arXiv preprint arXiv:2310.01238.

Zhang, F., Guo, T., & Wang, H. (2023). DFNet: Decomposition fusion model for long sequence time-series forecasting. *Knowledge-Based Systems*, *277*, 110794. https://doi.org/10.1016/j.knosys.2023.110794

Zhang, X. et al. (2025). Distribution shift aware neural tabular learning. arXiv preprint arXiv:2508.19486.

Zhou, Z., Yu, K.-Y., Guo, L.-Z., & Li, Y.-F. (2025). Fully test-time adaptation for tabular data. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 23027–23035). (*vol. 39*). https://doi.org/10.1609/aaai.v39i21.34466

Zhu, X., Zhao, L., & Zhu, W. (2024). Salience interest option: Temporal abstraction with salience interest functions. *Neural Networks*, *176*, 106342. https://doi.org/10.1016/j.neunet.2024.106342